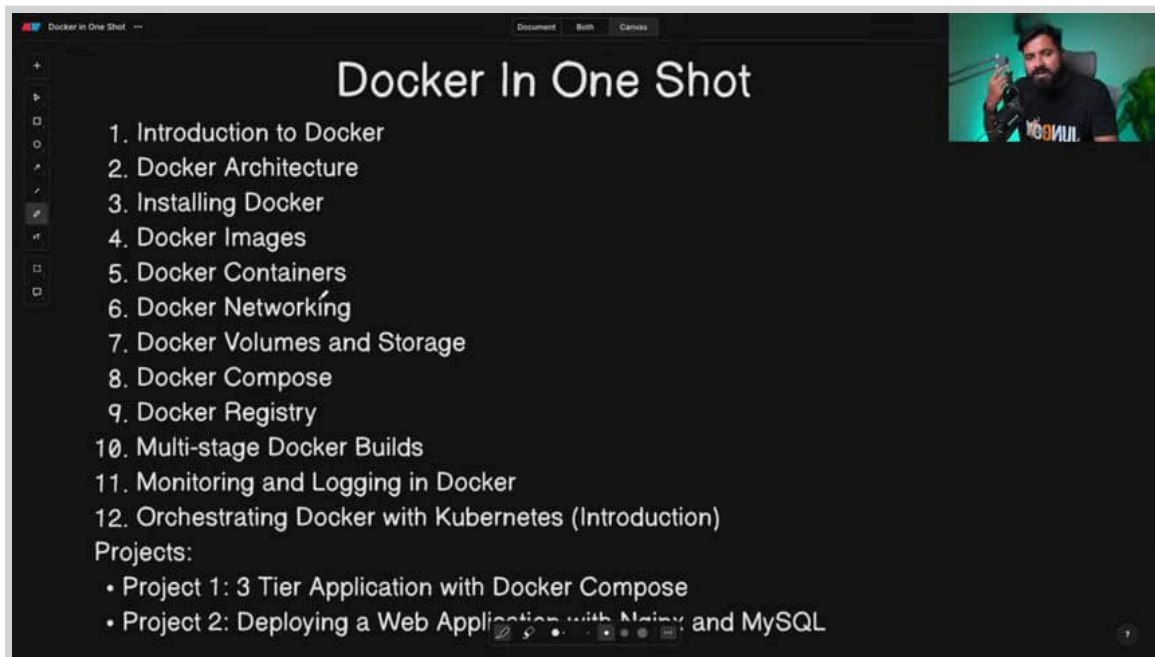


00:58



Introduction to Docker :- Docker is an **open-source containerization platform** that allows us to **package an application along with its dependencies, libraries, and configuration into a single unit called a container.**

These containers run the same way on **development, testing, and production environments**, which helps avoid the common problem of *"it works on my machine."*

We use Docker because it provides:

- **Consistency** across environments
- **Lightweight containers** (faster than virtual machines)
- **Faster application deployment**
- **Easy scalability and portability**
- **Better resource utilization**

In simple terms, Docker helps us **build once and run anywhere**, making application deployment faster, reliable, and efficient.



Virtualization vs Containerization

Virtualization

Virtualization means running **multiple Virtual Machines (VMs)** on a single physical server using a **hypervisor**.

Each VM has its **own full operating system**, along with applications and libraries.

Example: VMware, Hyper-V, KVM

Key points:

- Heavyweight (full OS per VM)
- Slower startup
- Strong isolation
- Uses more CPU, RAM, and storage

Containerization

Containerization means running **multiple containers** on a single OS using a **container engine like Docker**.

Containers share the **host OS kernel** and include only the application and its dependencies.

Example: Docker, Podman

Key points:

- Lightweight
- Very fast startup
- Uses fewer resources
- High portability

Difference Table (Interview Friendly)

Feature	Virtualization	Containerization
OS	Full OS per VM	Shared
host OS		
Boot Time	Minutes	Seconds
Performance	Slower	Near

nativeResource UsageHighLowIsolationStrongProcess-
levelPortabilityLimitedVery highScalingSlowFast

When to Use Virtualization?

Use **Virtual Machines** when:

- You need **different operating systems** on the same server (Linux + Windows)
- Strong **security and isolation** is required
- Running **legacy or monolithic applications**
- Kernel-level customization is needed



Example:

Running **Windows Server and Linux Server** together on Hyper-V.

When to Use Containerization?

Use **Containers** when:

- You want **fast deployment and scaling**
- Running **microservices**
- CI/CD pipelines
- Cloud-native applications
- Development & testing environments



Example:

Deploying a **Java or Node.js application** using Docker across dev, QA, and prod.

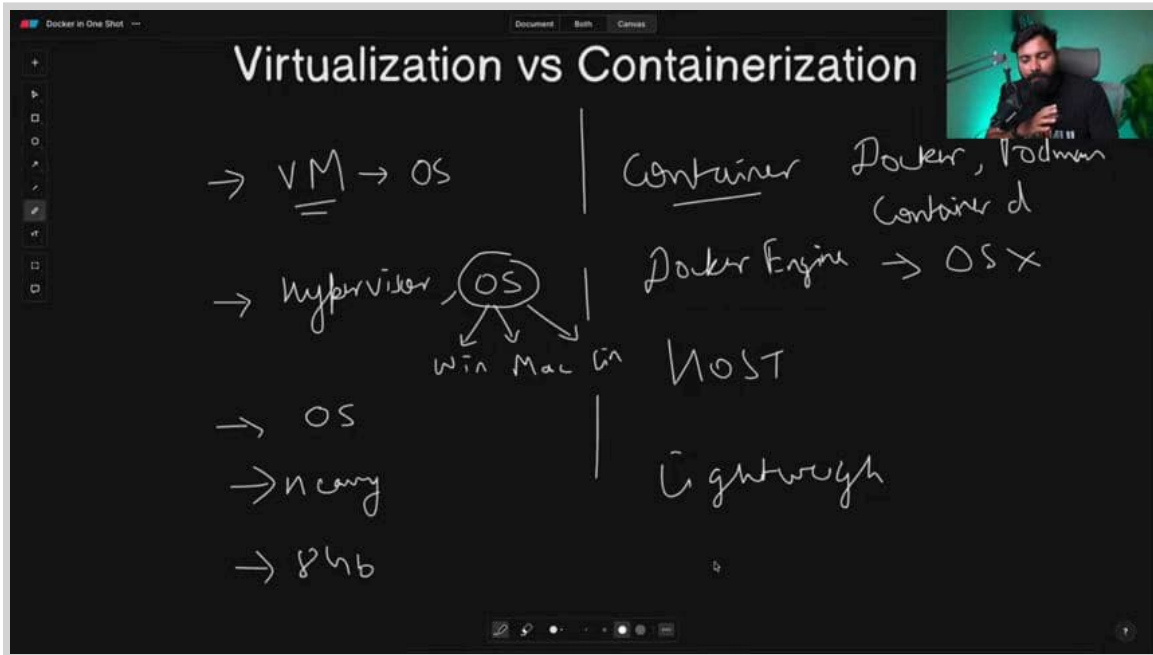
One-Line Interview Summary

Virtualization virtualizes hardware, while containerization virtualizes the operating system.

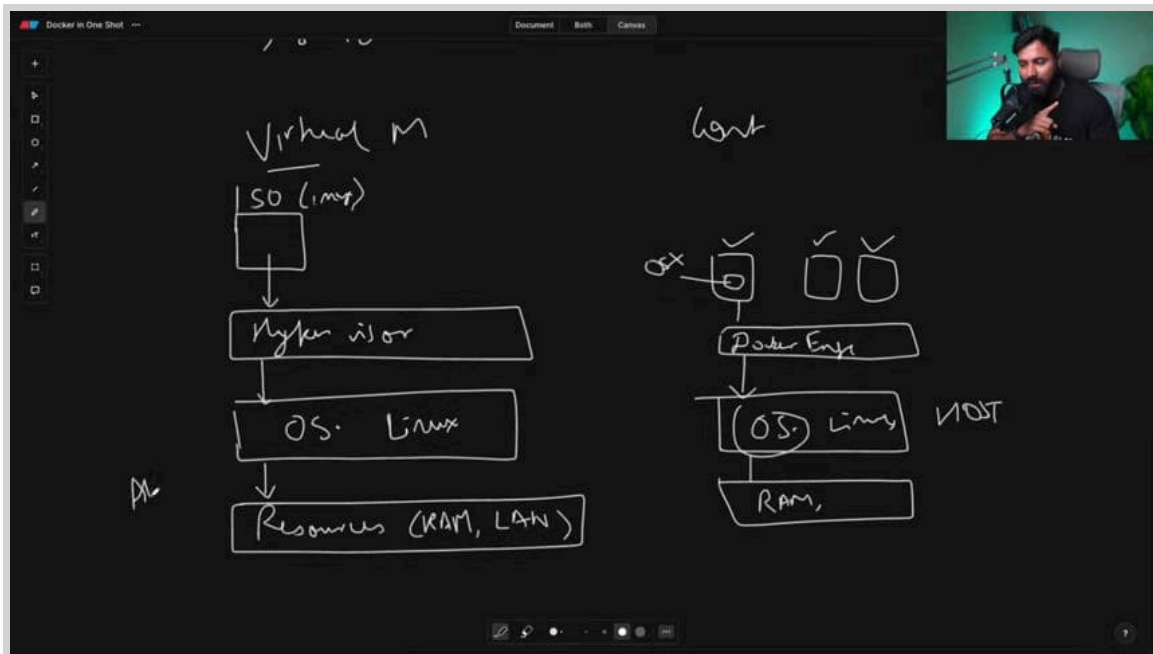
VMs are used when strong isolation and multiple OS are needed, and containers are used for fast, lightweight, and

scalable application deployment.

09:50



11:47



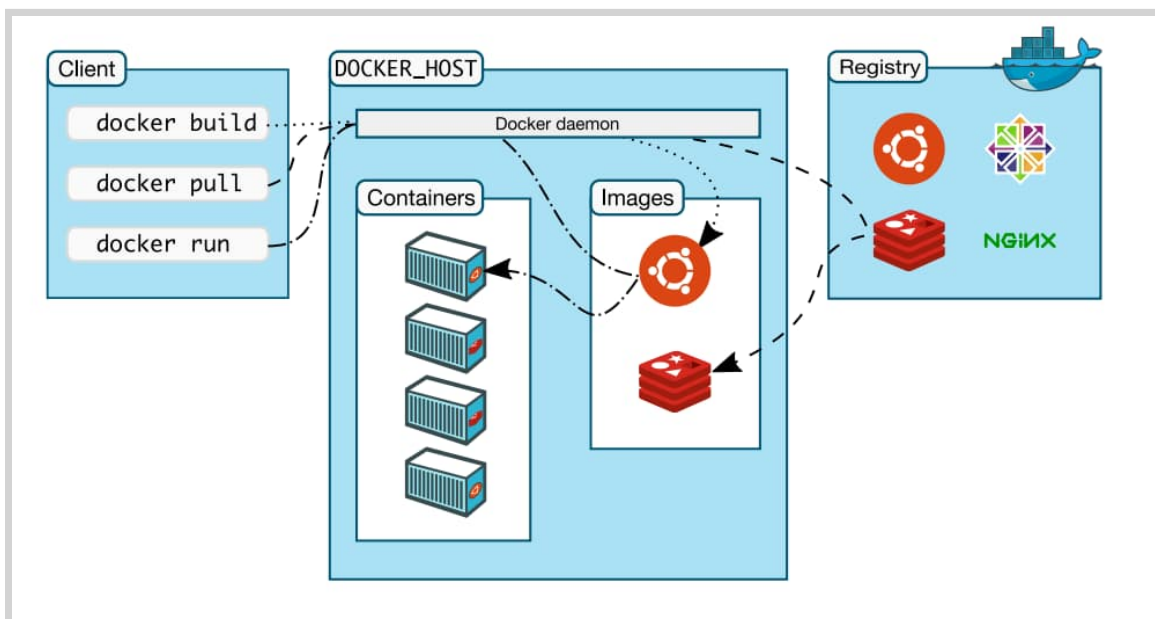
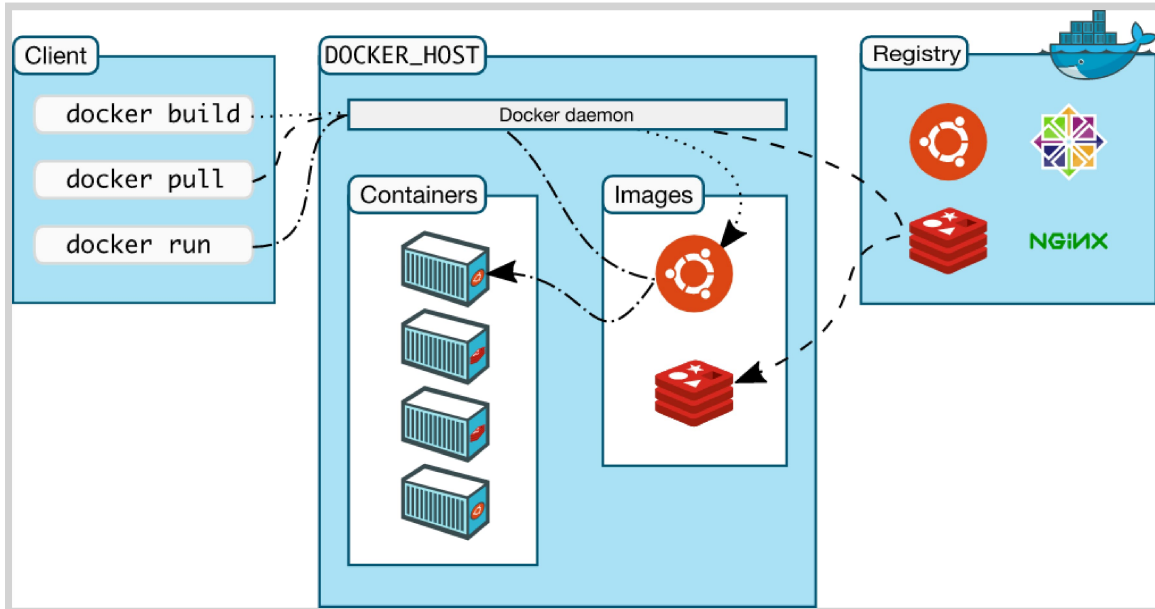
Docker Architecture:-

Docker Architecture

Docker follows a **Client-Server** architecture.

It mainly consists of **three core components**:

1. **Docker Client**
2. **Docker Daemon (Docker Engine)**
3. **Docker Registry**



<https://www.cherryservers.com/v3/assets/blog/2022-12-20/02.png>

1 Docker Client

The **Docker Client** is the interface used by the user to interact with Docker.

- Commands like:
 - `docker build`, `docker run`, `docker pull`, `docker push`
- The client sends these commands to the **Docker Daemon** using:
 - REST API
 - UNIX socket or TCP

 Example:

When you run `docker run nginx`, the client sends the request to the daemon.

2 Docker Daemon (dockerd)

The **Docker Daemon** is the **heart of Docker**.

It is responsible for:

- Building Docker images
- Creating and running containers
- Managing volumes and networks
- Communicating with Docker Registry

The daemon runs as a background service on the host machine.

 It uses:

- **Namespaces** → isolation
- **Cgroups** → resource control
- **Union File System** → image layers

3 Docker Registry

A **Docker Registry** is used to **store and distribute Docker images**.

- Public registry: **Docker Hub**
- Private registry: AWS ECR, GitLab Registry, Harbor

📌 Example:

- `docker pull nginx` → pulls image from Docker Hub
- `docker push myapp:v1` → pushes image to registry

Internal Components of Docker Engine

◆ Docker Images

- Read-only templates
- Built using Dockerfile
- Used to create containers

◆ Docker Containers

- Running instance of an image
- Isolated, lightweight, and fast

◆ Docker Volumes

- Used for **persistent storage**
- Data survives container deletion

◆ Docker Networks

- Enable container-to-container communication
- Types: bridge, host, overlay, none

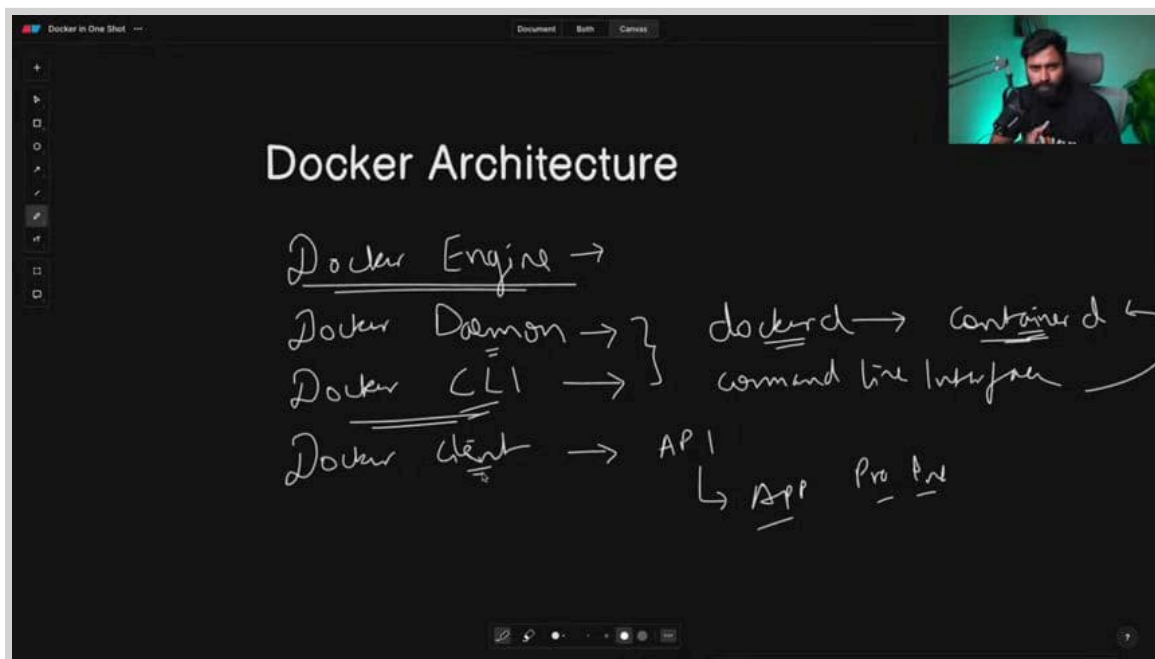
Docker Architecture Flow (Interview Explanation)

1. User runs a Docker command
 2. Docker Client sends request to Docker Daemon
 3. Daemon checks local image
 4. If image not found → pulls from Registry
 5. Daemon creates container using image
 6. Container runs on host OS kernel
-

One-Line Interview Answer

Docker uses a client-server architecture where the Docker client communicates with the Docker daemon, which manages images, containers, networks, and volumes, and pulls images from Docker registries.

15:31



Docker installation on Local:-

type Docker Desktop on computer and Create an account and download

Get docker---> download docker desktop according to your os

Docker Desktop ek esa client hai jo aap ko Docker engine ke saath baat karne deta hai

this practices would be done on AWS EC2 instance

After installation Docker on EC2 instance

Run Docker command on terminal , By this all commads will be visible on the screen that you can run with docker

=====

[26:49](#)

```
ubuntu@ip-172-31-7-73:~$ docker ps
permission denied while trying to connect to the Docker daemon socket at unix://var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/json": dial unix /var/run/docker.sock: connect: permission denied
ubuntu@ip-172-31-7-73:~$
```

solve this error

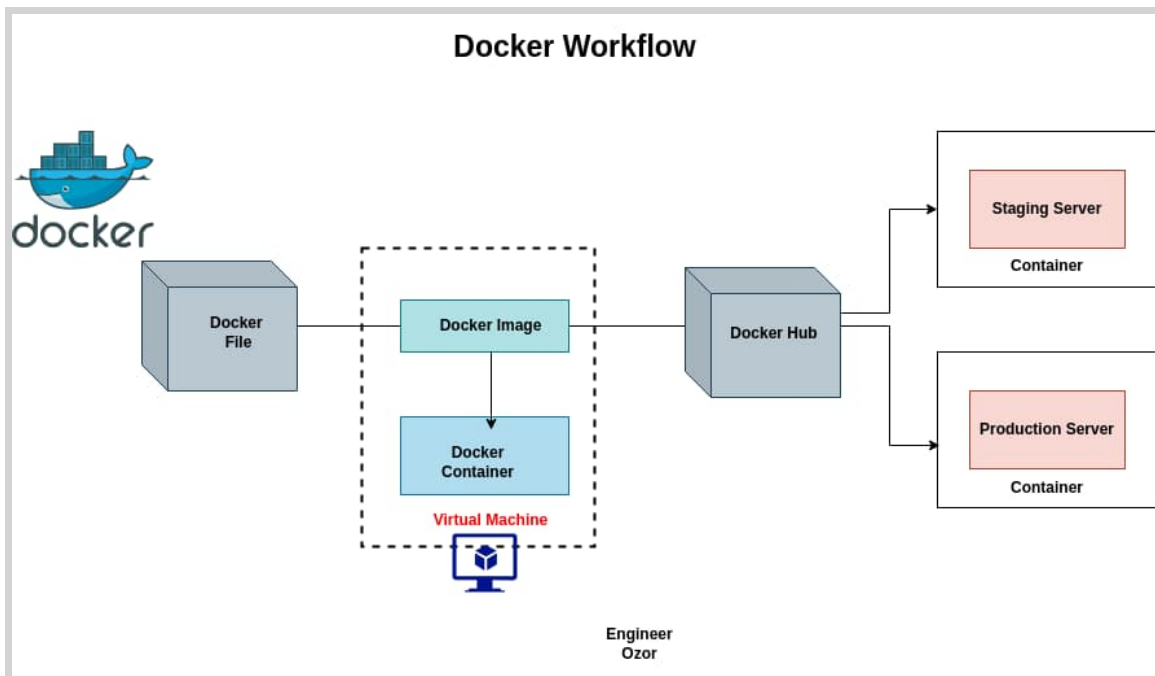
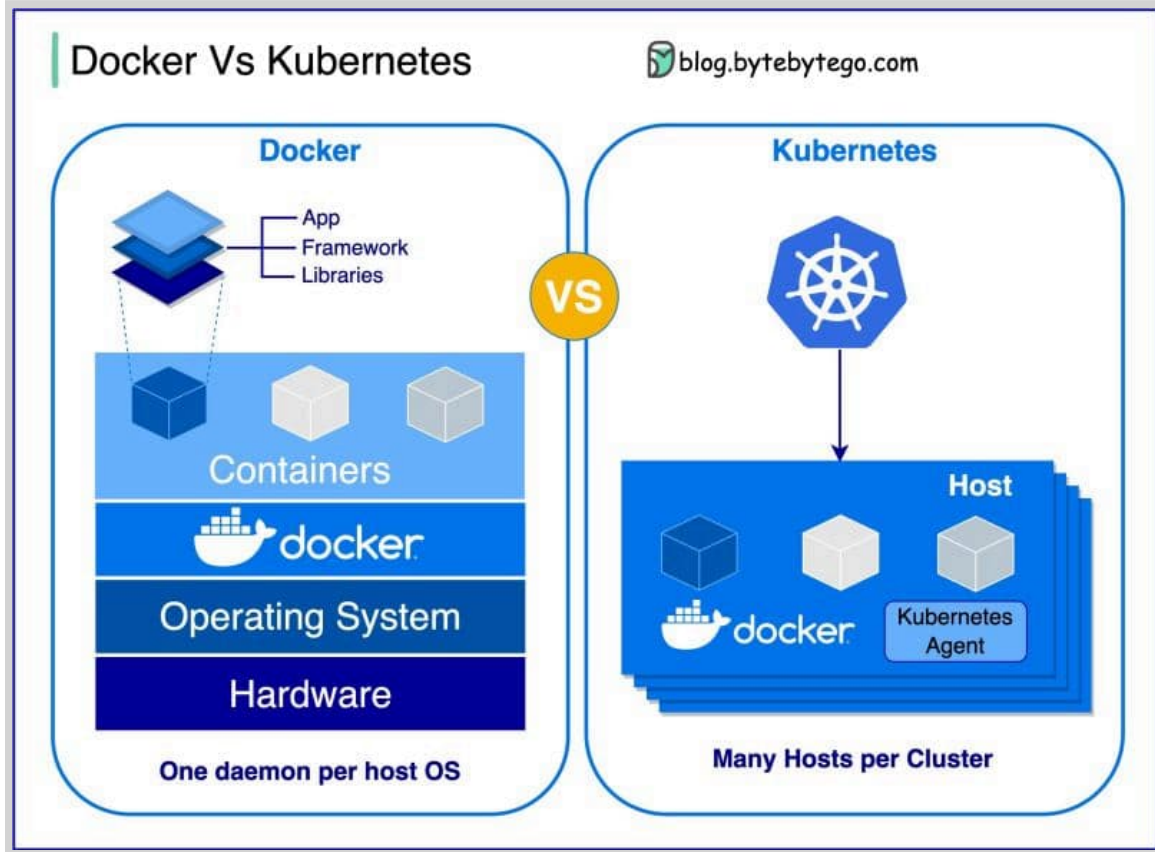
[28:06](#)

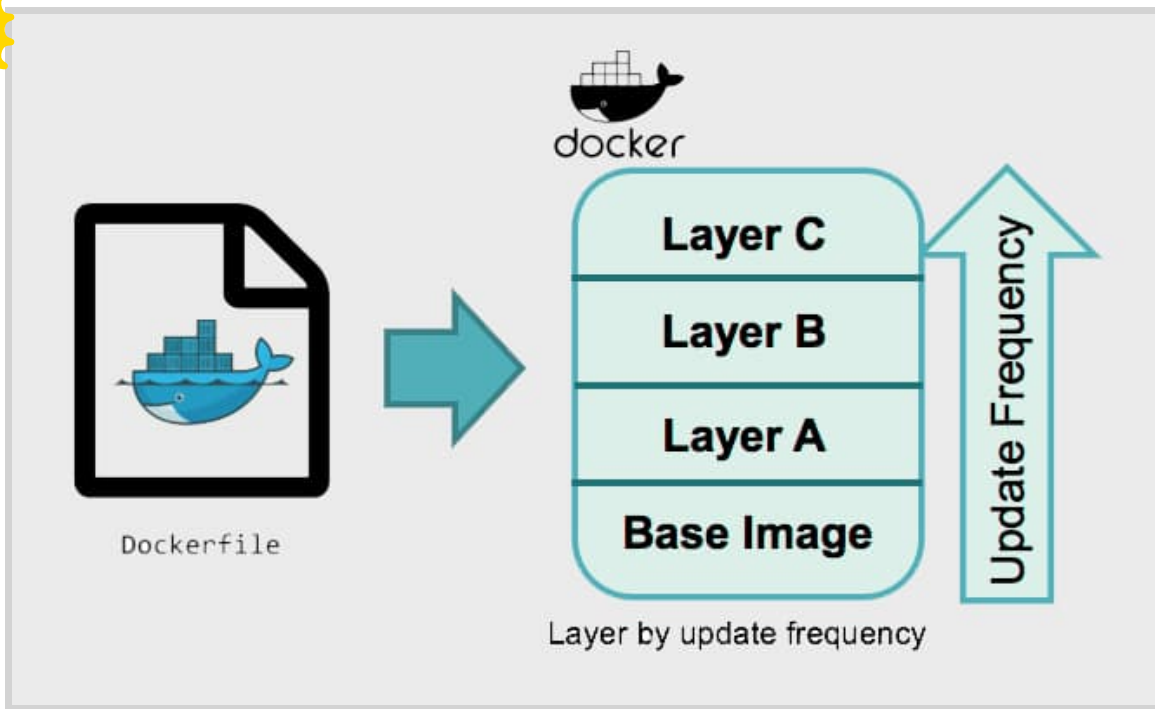
```
ubuntu@ip-172-31-7-73:~$ docker ps
permission denied while trying to connect to the Docker daemon socket at unix://var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/containers/json": dial unix /var/run/docker.sock: connect: permission denied
ubuntu@ip-172-31-7-73:~$ whoami
ubuntu
ubuntu@ip-172-31-7-73:~$ sudo usermod -aG docker $USER
ubuntu@ip-172-31-7-73:~$ docker ps
permission denied while trying to connect to the Docker daemon socket at unix://var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/containers/json": dial unix /var/run/docker.sock: connect: permission denied
ubuntu@ip-172-31-7-73:~$ newgrp docker
ubuntu@ip-172-31-7-73:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ubuntu@ip-172-31-7-73:~$
```

newgrp docker command se docker ko refresh kar diya
jisse jo changes kiye hai vo docker me apply ho gye

=====

Dockerfile





What is a Dockerfile?

A **Dockerfile** is a **text file** that contains a **set of instructions** used to **build a Docker image automatically**.

It defines:

- Base image
- Application code
- Dependencies
- Configuration
- Startup command

👉 In short: **Dockerfile = Blueprint to create Docker Image**

Basic Dockerfile Example

```
FROM ubuntu:22.04
RUN apt update && apt install -y nginx
COPY index.html /var/www/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```


Common Dockerfile Instructions (Very Important)

FROM

- Specifies the **base image**

```
FROM openjdk:17
```

RUN

- Executes commands **at build time**

```
RUN yum install -y httpd
```

COPY

- Copies files from host to container

```
COPY app.jar /app/
```

ADD

- Similar to COPY, but can extract tar files and download URLs
 - 👉 Interview tip: **Prefer COPY over ADD**
-

CMD

- Default command executed **when container starts**

```
CMD ["java", "-jar", "app.jar"]
```

ENTRYPOINT

- Makes the container behave like an executable

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

EXPOSE

- Documents the port used by the application

```
EXPOSE 8080
```

WORKDIR

- Sets working directory inside container

```
WORKDIR /app
```

ENV

- Sets environment variables

```
ENV APP_ENV=prod
```

CMD vs ENTRYPOINT (Must-Know)

CMDENTRYPOINTCan be overriddenHard to overrideDefault commandFixed commandUsed for flexibilityUsed for executables

📌 Best practice:

```
ENTRYPOINT ["java", "-jar"]
```

```
CMD ["app.jar"]
```

How Dockerfile Works (Build Flow)

1. Docker reads Dockerfile **top to bottom**
 2. Each instruction creates a **new image layer**
 3. Layers are cached
 4. Final image is created
-

Build Docker Image

```
docker build -t myapp:v1 .
```

Best Practices (Interview Bonus)

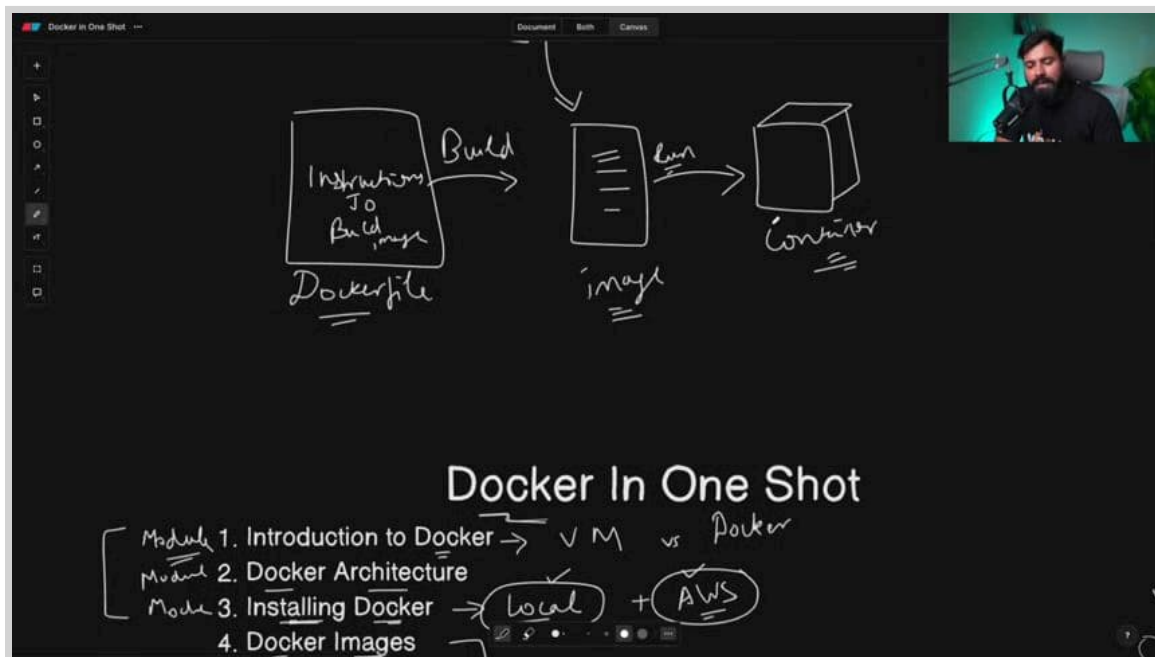
- Use **small base images** (alpine)
- Combine RUN commands to reduce layers
- Use `.dockerignore`
- Avoid running container as root

- Use multi-stage builds

One-Line Interview Answer

A Dockerfile is a text file that contains instructions to automatically build a Docker image, defining the application, dependencies, and startup command.

39:17



Docker Images:-Docker Images



DOCKER IMAGE LAYERS

www.learnitguide.net

1. Started Image Layer 3 as a container and accessible by users

1. Started Image Layer v1 as a container.
2. Installed and Configured https web server.
3. Committed new layer v2

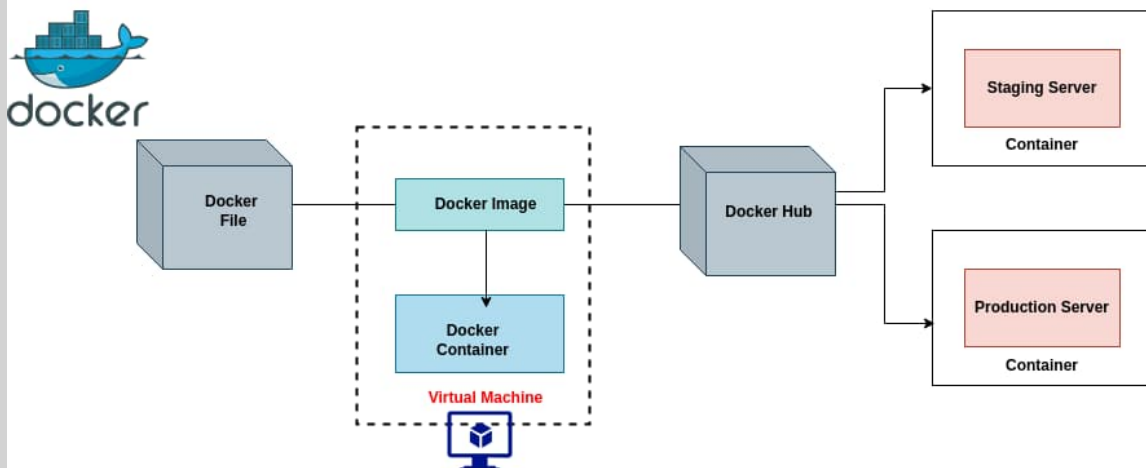
1. Started Base Image (**docker.io/centos**) as a container.
2. Package Updated on Base Image using "yum update".
3. Committed new layer v1

Pulled CentOS image from Docker Hub using docker pull command. Repo: **docker.io/centos**



<https://www.docker.com/app/uploads/2021/11/docker-containerized-and-vm-transparent-bg.png>

Docker Workflow



Engineer Ozor

What is a Docker Image?

A **Docker image** is a **read-only template** that contains everything required to run an application, such as:

- Application code

- Runtime (Java, Python, Node, etc.)
- Libraries & dependencies
- Environment variables
- Configuration files

A Docker image is used to **create containers**.

👉 **Image = Blueprint**

👉 **Container = Running instance of that blueprint**

Key Characteristics of Docker Images

- **Read-only** (cannot be modified once built)
- **Immutable**
- **Reusable**
- **Portable** (can run on any system with Docker)
- **Built in layers**

Docker Image Layers (Very Important)

Docker images are built in **multiple layers**.

- Each instruction in a **Dockerfile** creates a new layer
- Layers are **cached**
- Layers are **shared between images**

📌 Example:

```
FROM ubuntu
RUN apt install nginx
COPY app /var/www/html
```

Each line = one layer

- ✓ This makes builds faster
- ✓ Saves disk space

How Docker Images are Created



Method 1: Using Dockerfile

```
docker build -t myapp:v1 .
```

Method 2: Pull from Registry

```
docker pull nginx
```

Where Docker Images are Stored

- **Local system** (checked using `docker images`)
- **Docker Registry**
 - Public: Docker Hub
 - Private: AWS ECR, GitLab Registry, Harbor

Docker Image vs Docker Container

Docker Image	Docker Container
Static	Running
Read-only	Read-write
Blueprint	Instance
Cannot run	Runs application

Common Docker Image Commands

<code>docker images</code>	<code># list images</code>
<code>docker pull nginx</code>	<code># download image</code>
<code>docker build -t app:v1 .</code>	<code># create image</code>
<code>docker rmi image_id</code>	<code># delete image</code>
<code>docker tag app:v1 app:v2</code>	<code># tag image</code>
<code>docker push app:v1</code>	<code># push to registry</code>

Real-World Example (Interview Gold)

In my project, we created a Docker image for a Java application using OpenJDK as the base image.

The image was built using a Dockerfile and pushed to a private registry.

The same image was used across dev, QA, and production to maintain consistency.

How to pull prebuild container images:-

33:54

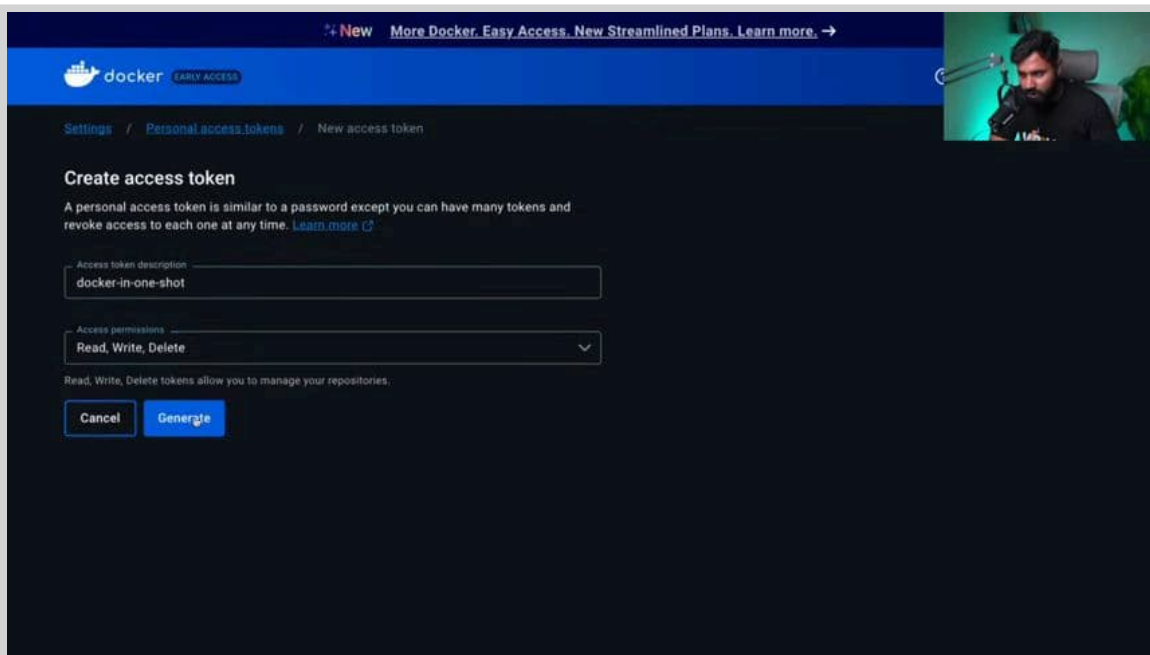
```
ubuntu@ip-172-31-7-73:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS
ubuntu@ip-172-31-7-73:~$ docker images
REPOSITORY    TAG       IMAGE ID   CREATED   SIZE
ubuntu@ip-172-31-7-73:~$ docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com/ to create one.
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better security and is required for organizations using SSO. Learn more at https://docs.docker.com/go/access-tokens/

Username: trainwithshubham
Password:
Error response from daemon: Get "https://registry-1.docker.io/v2/": unauthorized: incorrect username or password
ubuntu@ip-172-31-7-73:~$
```

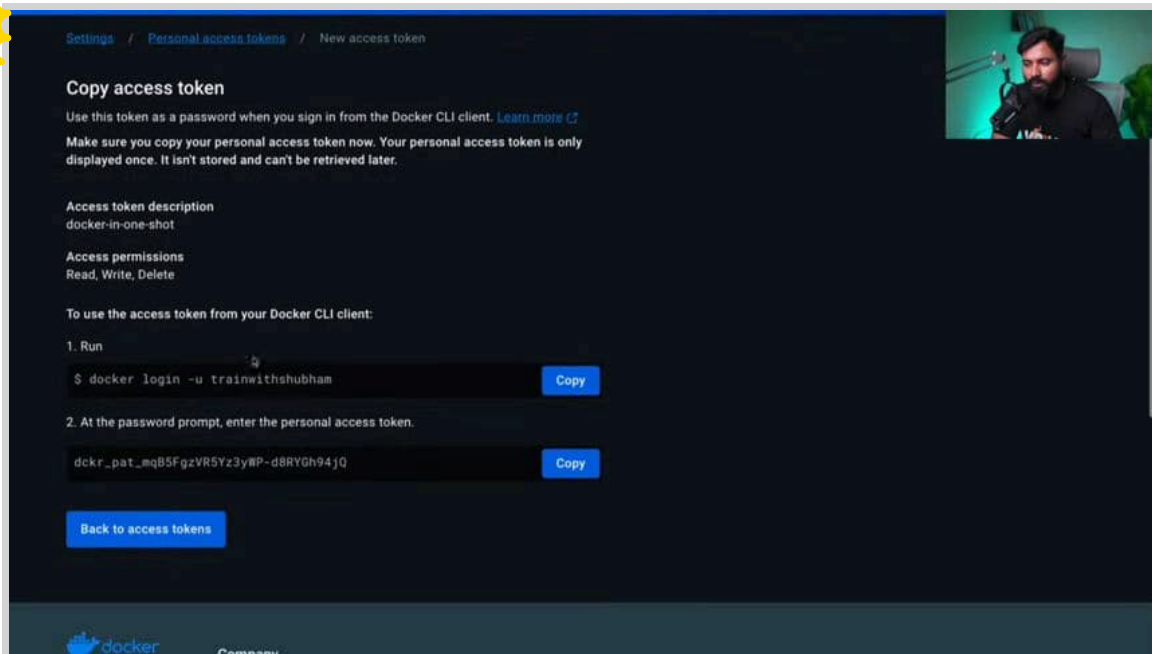
Login Docker By Using PAT(Personal Access Token) :-

Go to Docker Hub ==> Account settings ==> PAT (Personal Access Token) ==> Generate New Token

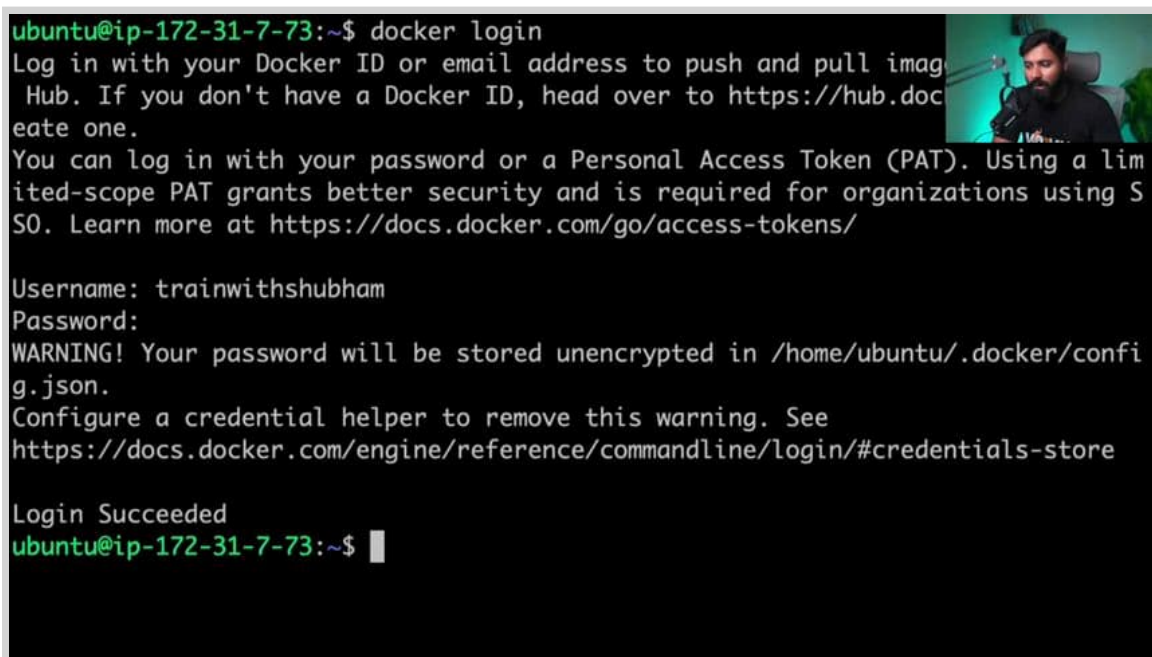
35:02



35:05



35:37



37:01

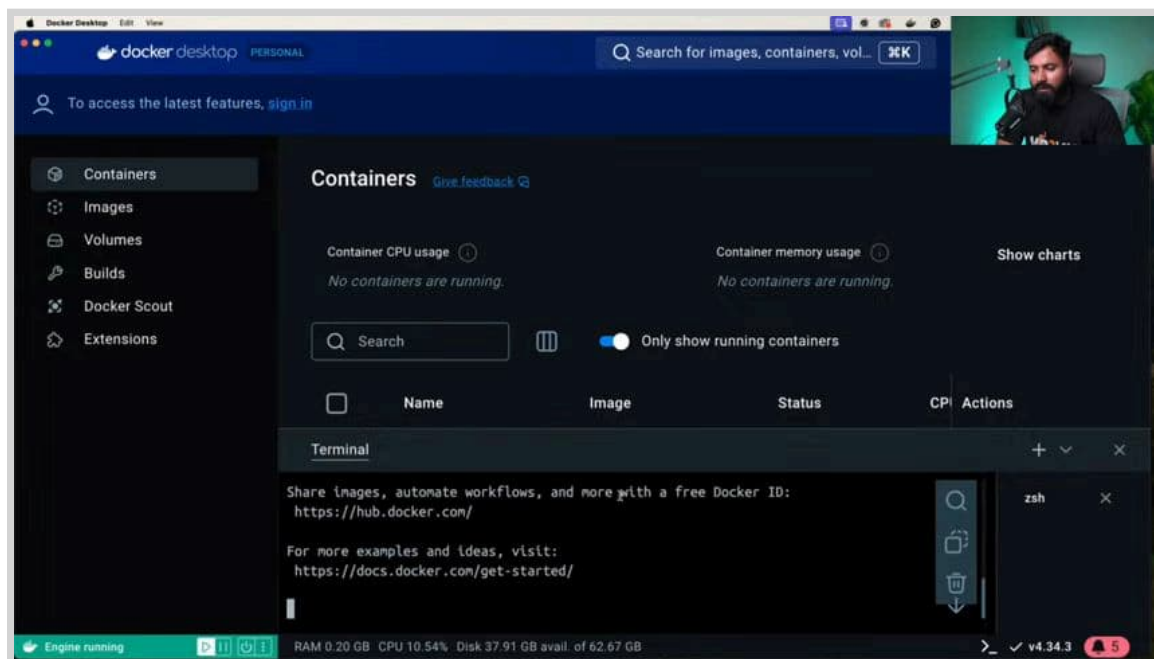
```
ubuntu@ip-172-31-7-73:~$ docker pull hello-world
Using default tag: latest
latest: Pulling from library/hello-world
c1ec31eb5944: Pull complete
Digest: sha256:d211f485f2dd1dee407a80973c8f129f00d54604d2c90732e8e320e5038a0348
Status: Downloaded newer image for hello-world:latest
docker.io/library/hello-world:latest
ubuntu@ip-172-31-7-73:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world    latest    d2c94e258dcb   18 months ago 13.3kB
ubuntu@ip-172-31-7-73:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
```

if you direct run command `docker run hello-world` on terminal firstly he pull images and then after Run container Here you not need to pull docker images.

[37:33](#)



To pull mysql images

[38:35](#)

```
ubuntu@ip-172-31-7-73:~$ docker pull mysql
Using default tag: latest
latest: Pulling from library/mysql
eba3c26198b7: Pull complete
97f7c8c33abe: Pull complete
aa23d877fa04: Pull complete
a143609ddd2d: Pull complete
78308a3437c4: Pull complete
c0880e4b3737: Pull complete
4bab267f9ce1: Pull complete
e575f6d9b17a: Pull complete
607f86c00053: Extracting 26.74MB/68.74MB
cd68caa5febe: Download complete
```



mysql ek server hota hai use access karne ke liye username password chahiye hota hai

[39:58](#)

```
ubuntu@ip-172-31-7-73:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
mysql          latest   be960704dfac   3 weeks ago    602MB
hello-world    latest   d2c94e258dcb   18 months ago  13.3kB
ubuntu@ip-172-31-7-73:~$ docker run -e MYSQL_ROOT_PASSWORD=root mysql
```

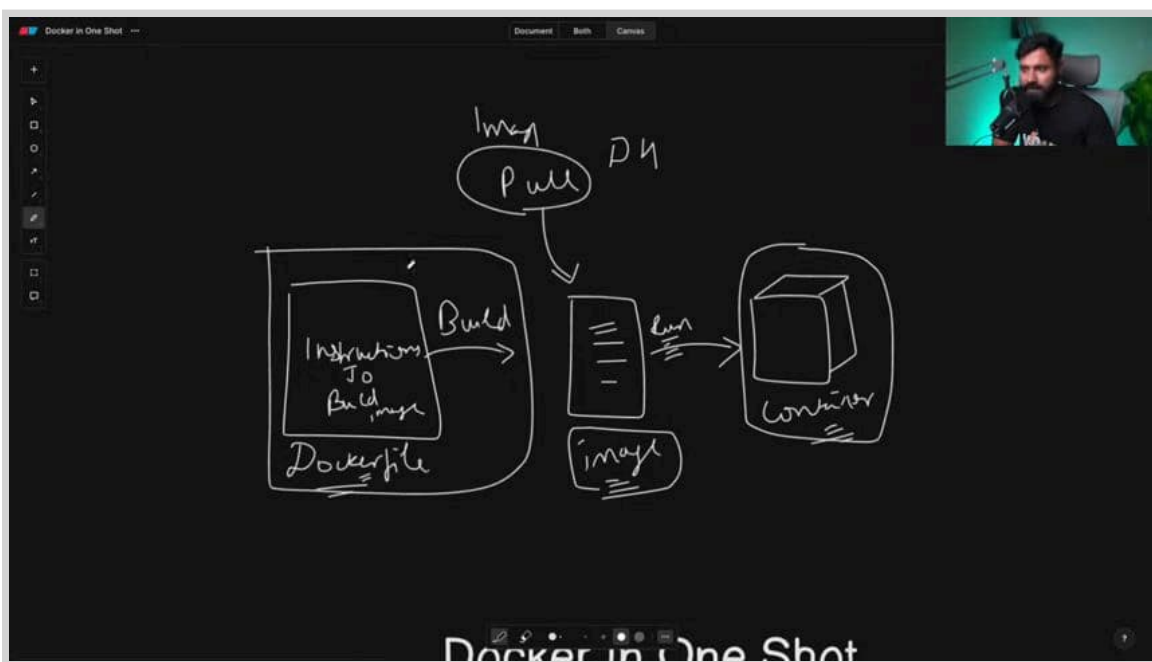


[42:24](#)

```
ubuntu@ip-172-31-7-73:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
5             NAMES
fb293b16554f   mysql    "docker-entrypoint.s..." About a minute ago Up About
/tcp, 33060/tcp vigilant_napier
ubuntu@ip-172-31-7-73:~$ docker stop fb293b16554f
```

to Run container in background or detach mode

[44:05](#)



[43:07](#)

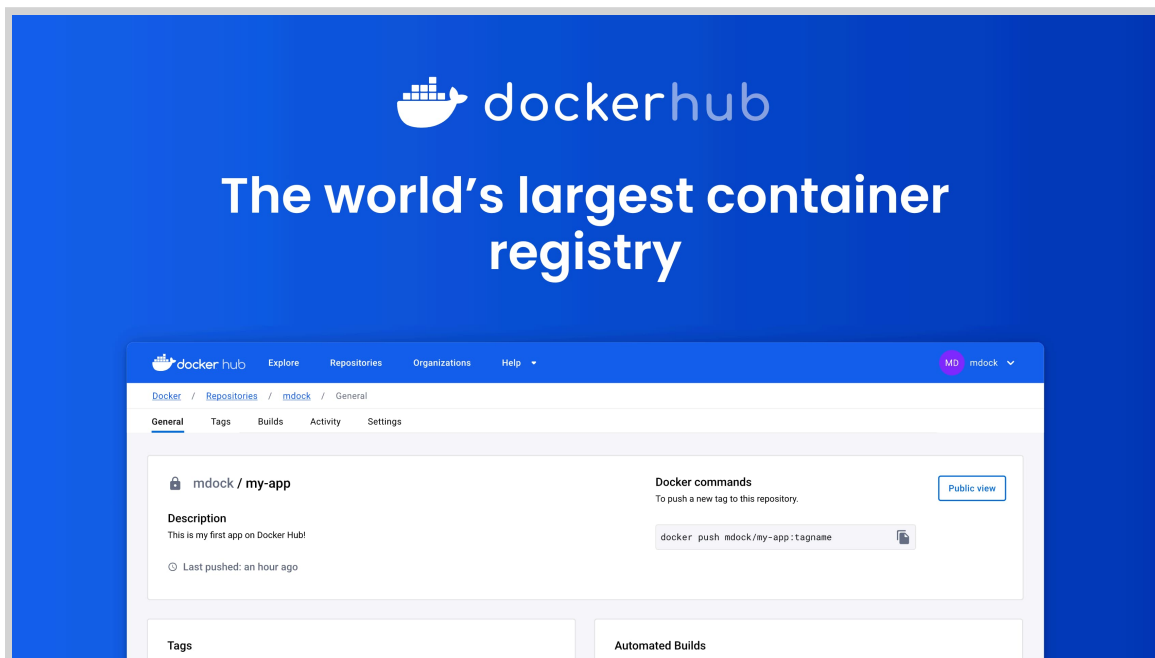
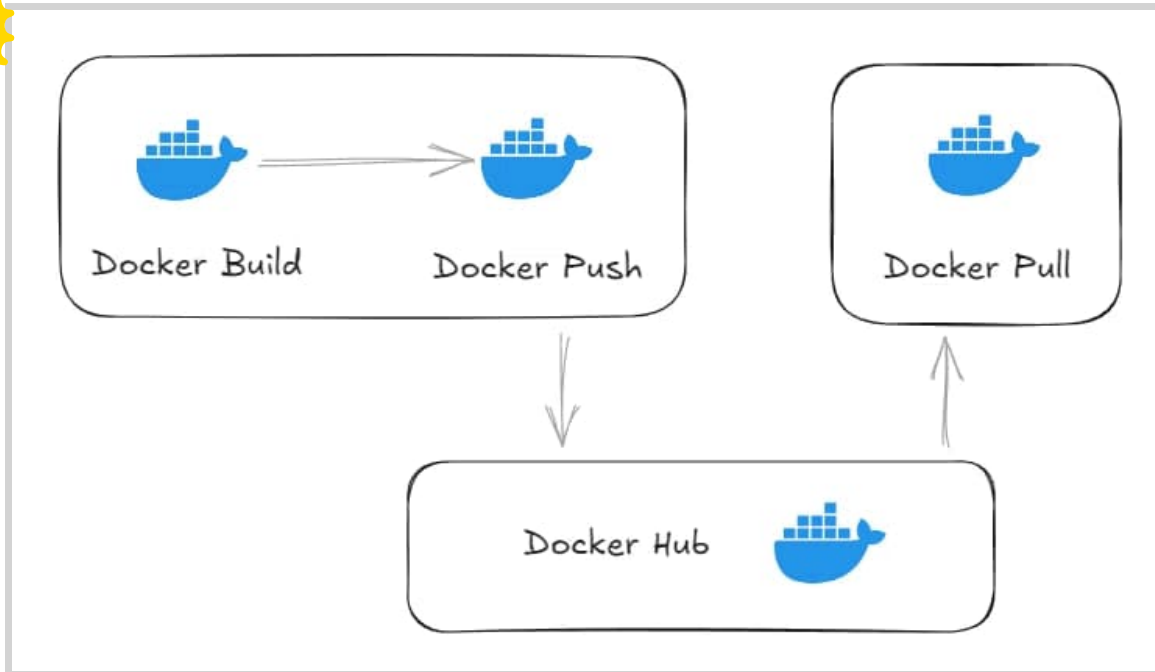
```
ubuntu@ip-172-31-7-73:~$ ls
ubuntu@ip-172-31-7-73:~$ docker run -d -e MYSQL_ROOT_PASSWORD=root
```



=====

Docker Hub

 <https://www.docker.com/app/uploads/2022/06/Screen-Shot-2020-06-08-at-6.03.35-PM.png>



What is Docker Hub?

Docker Hub is a cloud-based container image registry used to store, share, and manage Docker images.

It works like **GitHub** for Docker images.

Why Do We Use Docker Hub?

Docker Hub is used to:

- Store Docker images centrally
- Pull ready-made images (nginx, mysql, java, etc.)
- Share images between teams
- Integrate with CI/CD pipelines
- Deploy applications faster

📌 Example:

```
docker pull nginx
```

This pulls the nginx image from Docker Hub.

Types of Images in Docker Hub

◆ Official Images

- Maintained by Docker
- Secure and optimized
- Examples: nginx, mysql, httpd, openjdk

◆ User Images

- Created and maintained by users or organizations
- Example: mycompany/myapp:v1

Basic Docker Hub Workflow

1. Developer creates Docker image
2. Image is tagged
3. Image is pushed to Docker Hub
4. Other systems pull the image and run containers

Important Docker Hub Commands

```
docker login                                # login to
Docker Hub
docker tag myapp:v1 user/myapp:v1
docker push user/myapp:v1                  # upload image
```




`docker pull user/myapp:v1`
image

download

Public vs Private Repositories

Public Repo	Private Repo	Anyone can pull	Restricted
access	Free	Limited free / Paid	Open source images
			Company apps

Real-World Example (Interview Gold)

In our project, we built Docker images using a Dockerfile and pushed them to Docker Hub.

Jenkins pulled the image from Docker Hub and deployed containers on the production server.

Security Features

- Image scanning
 - Access control
 - Tags & versioning
 - Private repositories
 - Webhooks
-

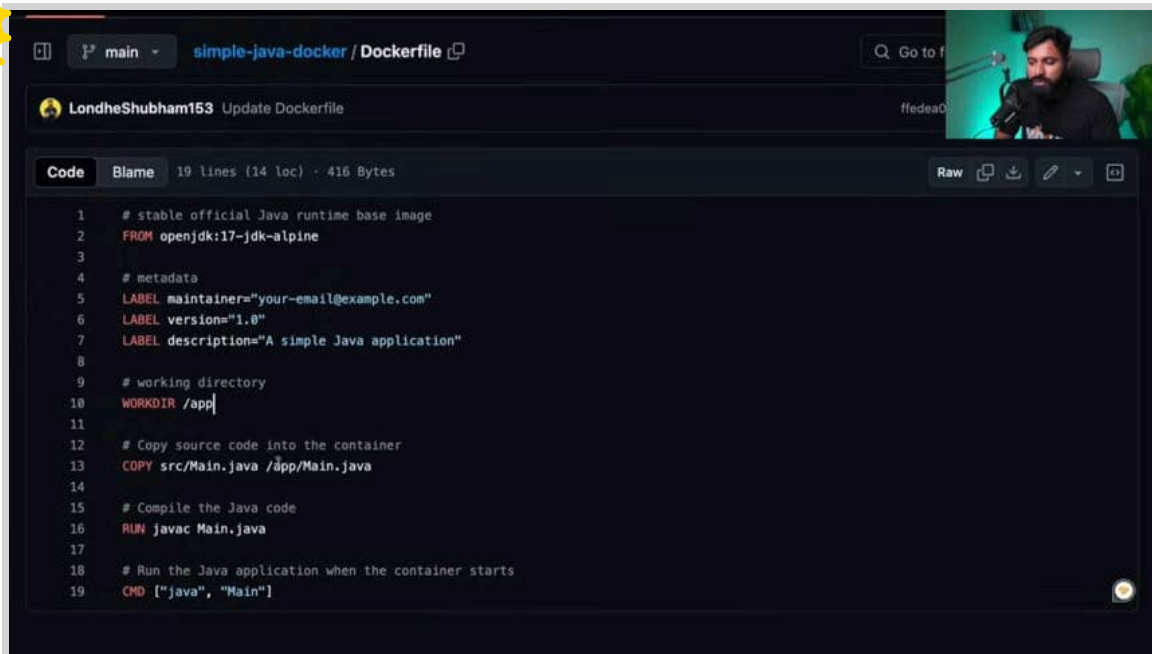
One-Line Interview Answer

Docker Hub is a cloud-based registry service used to store, share, and distribute Docker images.

=====

Create Docker file for java :- Docker file me kuch instrauction hote hai jiski wajah se aapka application chalta hai is video ka minute 42 se 60 minutes ka suno ache se ==> Search simaple-project-for-java on git clone that Git project from online repository to local machine

[46:31](#)

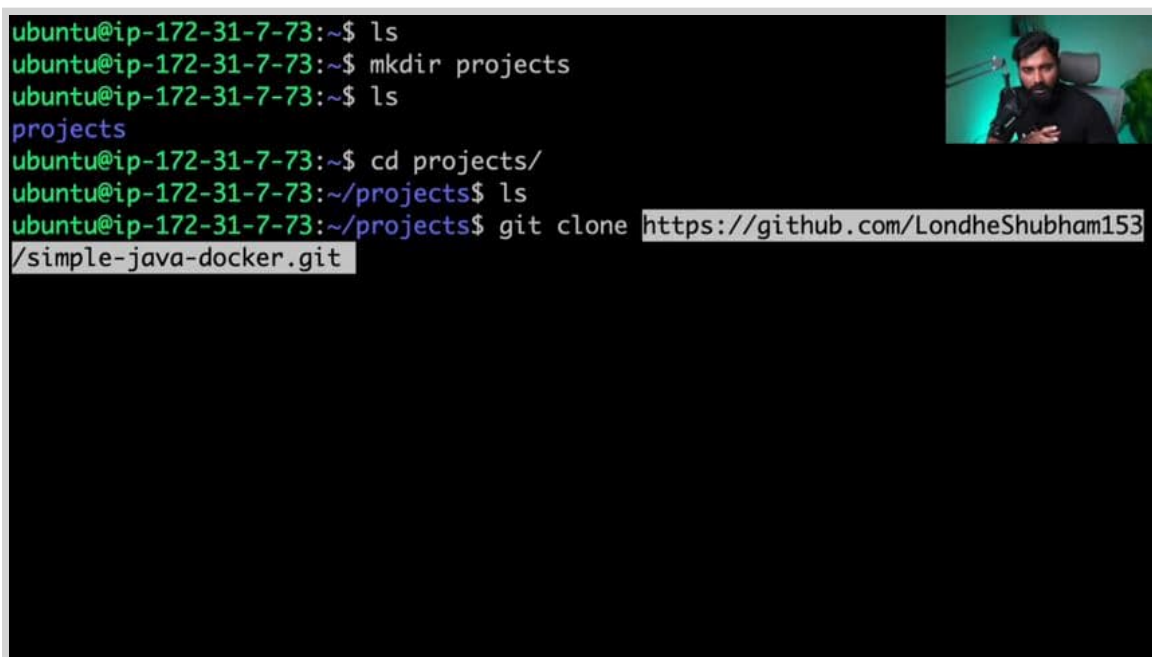


```

1  # stable official Java runtime base image
2  FROM openjdk:17-jdk-alpine
3
4  # metadata
5  LABEL maintainer="your-email@example.com"
6  LABEL version="1.0"
7  LABEL description="A simple Java application"
8
9  # working directory
10 WORKDIR /app
11
12 # Copy source code into the container
13 COPY src/Main.java /app/Main.java
14
15 # Compile the Java code
16 RUN javac Main.java
17
18 # Run the Java application when the container starts
19 CMD ["java", "Main"]

```

45:36



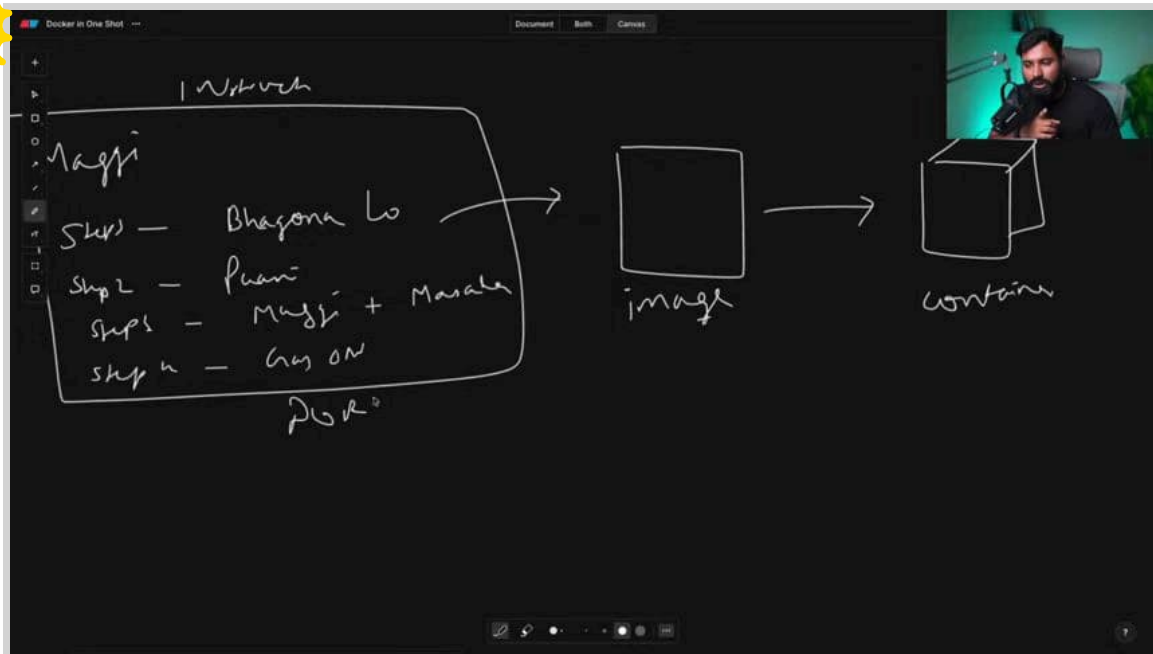
```

ubuntu@ip-172-31-7-73:~$ ls
ubuntu@ip-172-31-7-73:~$ mkdir projects
ubuntu@ip-172-31-7-73:~$ ls
projects
ubuntu@ip-172-31-7-73:~$ cd projects/
ubuntu@ip-172-31-7-73:~/projects$ ls
ubuntu@ip-172-31-7-73:~/projects$ git clone https://github.com/LondheShubham153/simple-java-docker.git

```

==> Dockerfile Create for it :-
file creation yaad karne ki trick:-

48:39



sabse pahale java applicatio ke liye environment chahiye hoga (patila chahiye hoga) jisme java already install hai, jaise meggin banane ke liye aap ko chahiye patila vese hi java ki application chalane ke liye ek baseimage use karenge

==> FROM openjdk:17-jdk-alpine

":" ke baad version aaata hai agar nhi lihoge to vo latest version utha lega

==> ab aap ko working directory banani hai

aap ko is container me ek esa folder banana hai jisme vo application source code rakha rhe

WORKDIR /app

==> ab me isme copy kar luga , har vo chiz ho mere host me rakhi hui hai vo mere container me copy kar luga

COPY src/main.java /app/main.java

==> Ab is java ka code ko run karne ke liye pahale compile karna padta hai to me chata hu is jab code copy ho jaate to ye javac command run ho to iske liye , RUN ka use karuga

RUN javac main.java

=====> yaha tak ki commands kisi intermindiate container ko banane ke liye hoti hai iske baad container ko chalne ke aap jo uper se aap daalte ho vo CMD hai taki container banane ke baad vo container excute ho jaayega agar CMD nhi likhoge to bhi container banega lekin CMD lekhne ke baad vo container excute bhi ho jaayega ==> ab me application ko run karne ke liye CMD ["java", "main"]

[57:47](#)

```
# pull a base image which gives all required tools and libraries
FROM openjdk:17-jdk-alpine

# create a folder where the app code will be stored
WORKDIR /app

# Copy the source code from your HOST machine to your container
COPY src/Main.java /app/Main.java

# compile the application code
RUN javac Main.java

# run the application
CMD ["java", "Main"]

-- INSERT --
```



15,20 All

[57:55](#)

```
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ vim Dockerfile
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ cat Dockerfile
# pull a base image which gives all required tools and libraries
FROM openjdk:17-jdk-alpine

# create a folder where the app code will be stored
WORKDIR /app

# Copy the source code from your HOST machine to your container
COPY src/Main.java /app/Main.java

# compile the application code
RUN javac Main.java

# run the application
CMD ["java","Main"]
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$
```



[58:42](#)

```
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ docker build -t java-app .
```



"." ka mtlab context , context mtlab Dockerfile , mtlab Dockerfile
ho context lekar ye image ban rhi hai
or Dockerfile rkhi hui kaha hai isi directory me iseliye "."

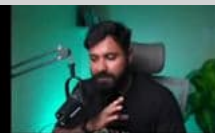
[59:23](#)

```
Step 1/5 : FROM openjdk:17-jdk-alpine
17-jdk-alpine: Pulling from library/openjdk
5843afab3874: Pull complete
53c9466125e4: Pull complete
d8d715783b80: Pull complete
Digest: sha256:4b6abae565492dbe9e7a894137c966a7485154238902f2f25e9dbd9784383d81
Status: Downloaded newer image for openjdk:17-jdk-alpine
---> 264c9bdce361
Step 2/5 : WORKDIR /app
---> Running in 714c67c639ef
Removing intermediate container 714c67c639ef
---> 2d6561ea5299
Step 3/5 : COPY src/Main.java /app/Main.java
---> 1dfc2430c324
Step 4/5 : RUN javac Main.java
---> Running in c18bffe75a6b
Removing intermediate container c18bffe75a6b
---> cfd9b02b2cfe
Step 5/5 : CMD ["java","Main"]
```



[59:26](#)

```
Status: Downloaded newer image for openjdk:17-jdk-alpine
---> 264c9bdce361
Step 2/5 : WORKDIR /app
---> Running in 714c67c639ef
Removing intermediate container 714c67c639ef
---> 2d6561ea5299
Step 3/5 : COPY src/Main.java /app/Main.java
---> 1dfc2430c324
Step 4/5 : RUN javac Main.java
---> Running in c18bffe75a6b
Removing intermediate container c18bffe75a6b
---> cfd9b02b2cfe
Step 5/5 : CMD ["java","Main"]
---> Running in 3c55c8277110
Removing intermediate container 3c55c8277110
---> 3b0f735721e5
Successfully built 3b0f735721e5
Successfully tagged java-app:latest
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$
```



[59:58](#)


```
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ docker image
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
java-app      latest    3b0f735721e5   32 seconds ago 326
mysql         latest    be960704dfac   3 weeks ago    602
hello-world   latest    d2c94e258dcb   18 months ago 13.3kB
openjdk       17-jdk-alpine 264c9bdce361   3 years ago    326MB
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ docker run java-app
Hello, Docker! Current date: Wed Nov 06 09:48:51 GMT 2024
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$
```

ab mene java ke code me kuch changes kiye hai or file ko save kiya hu

[01:00:47](#)

```
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ docker image
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
java-app      latest    3b0f735721e5   32 seconds ago 326
mysql         latest    be960704dfac   3 weeks ago    602
hello-world   latest    d2c94e258dcb   18 months ago 13.3kB
openjdk       17-jdk-alpine 264c9bdce361   3 years ago    326MB
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ docker run java-app
Hello, Docker! Current date: Wed Nov 06 09:48:51 GMT 2024
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ vim src/Main.java
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ docker run java-app
```

==>jab bhi code me chage karoge to image ko bhi update karne hoga
uske baad jo container banega vo updated image ka container hoga

[01:01:22](#)


```
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ docker image
REPOSITORY    TAG          IMAGE ID      CREATED       SIZE
java-app      latest      3b0f735721e5  32 seconds ago 326MB
mysql         latest      be960704dfac  3 weeks ago   602MB
hello-world   latest      d2c94e258dcb  18 months ago 13.3kB
openjdk       17-jdk-alpine 264c9bdce361  3 years ago   326MB
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ docker run java-app
Hello, Docker! Current date: Wed Nov 06 09:48:51 GMT 2024
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ vim src/Main.java
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ docker run java-app
Hello, Docker! Current date: Wed Nov 06 09:49:44 GMT 2024
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ docker build -t java-app
```

[01:01:56](#)

```
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$ docker run j
Hello, Dosto! Subscribe kar lo, Current date: Wed Nov 06 09:50:4
ubuntu@ip-172-31-7-73:~/projects/simple-java-docker$
```

=====

Create Python Dockerfile :-

vahi browser se koi project local machine par clone kar lo

[01:04:11](#)

```
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ ls
README.md app.py requirements.txt run.py
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ cat requirements.
flask==2.2.2ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$
```

==>requirements me likha hua hai flask ka version 2.2 chahiye to create

==>vim Dockerfile

==> to create environment for python

FROM python:3.7

==> to create workdir in which you want this container should be run

WORKDIR /app

==> copy soruce code into your container

COPY . .

==> install dependecy or lybrari that in required before create container.

RUN pip install requirement.txt

==> excecute container by using CMD command or by using

ENTRYPOINT

ENTRYPOINT ["python","run.py"]

=====

Difference Between CMD and ENTRYPOINT

 https://www.docker.com/app/uploads/2024/07/2400x1260_run-cmd-entrypoint-1110x583.png

```
demo -- -bash -- 97x16
abhinav:demo abhi$ docker run entrypoint-instructions printenv
Hello World printenv
abhinav:demo abhi$
abhinav:demo abhi$
abhinav:demo abhi$ docker run --entrypoint printenv entrypoint-instructions
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=8ae980d69b12
HOME=/root
abhinav:demo abhi$
```

 https://www.docker.com/app/uploads/2024/07/2400x1260_decision-tree-exec-vs-shell-1110x583.png

CMD

- Provides the **default command** to run when a container starts
- Can be **easily overridden** from the command line
- Used when you want **flexibility**

CMD ["nginx", "-g", "daemon off;"]

docker run myimage bash

CMD is overridden

ENTRYPOINT

- Defines the **main command** that always runs
- **Cannot be easily overridden**
- Used when the container should behave like an **executable**

```
ENTRYPOINT ["nginx", "-g", "daemon off;"]
```

```
docker run myimage bash
```

```
# bash is passed as argument, nginx still runs
```

CMD vs ENTRYPOINT (Table)

Feature	CMD	ENTRYPOINT	Purpose	Default command	Fixed
main command	Override	Easy	Hard	Use case	Flexible
containers	Executable	containers	CLI behavior	Replaced by	
command	Command is appended as argument				

Best Practice (Very Important)

Use **ENTRYPOINT + CMD** together:

```
ENTRYPOINT ["java", "-jar"]
```

```
CMD ["app.jar"]
```

- 👉 ENTRYPOINT → fixed command
- 👉 CMD → default argument (can be changed)

```
docker run myimage other.jar
```

Interview One-Line Answer

CMD provides a default command that can be overridden, whereas ENTRYPOINT defines a fixed command that always runs when the container starts.

Real-World Example

- **CMD** → development images (debugging, testing)
- **ENTRYPOINT** → production images (apps, services)

=====

01:07:14

```
FROM python:3.7

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

ENTRYPOINT ["python", "run.py"]

-- INSERT --
```

9,31 All

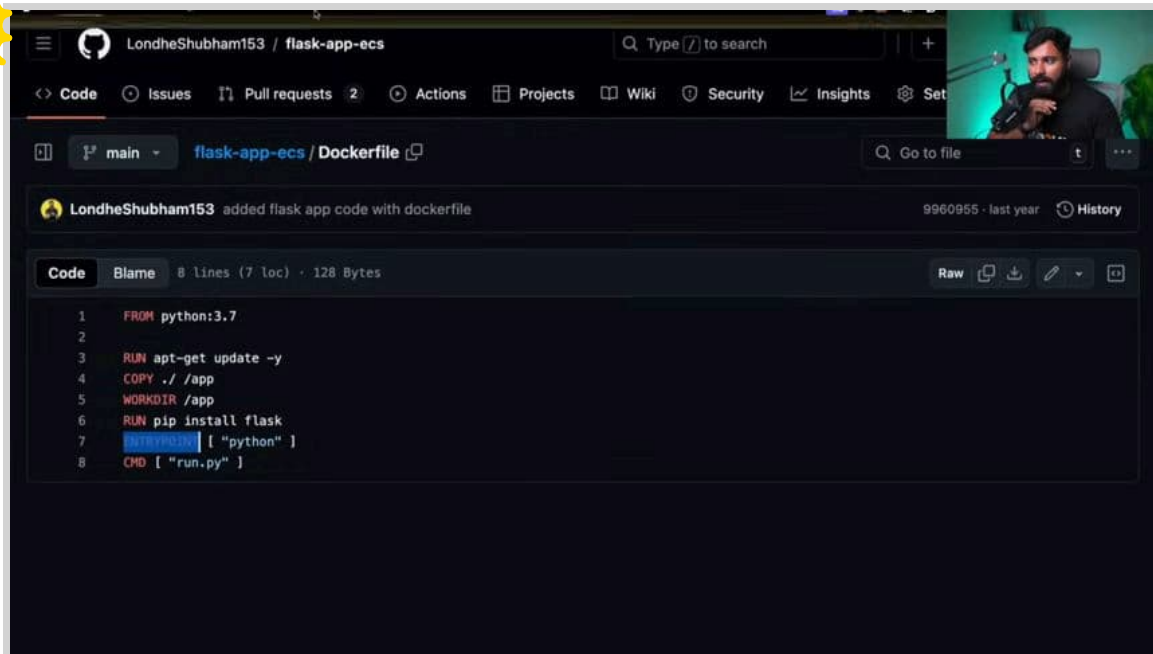
01:07:34

```
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ vim Dockerfile
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ docker build -t flask-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.

                Install the buildx component to build images with BuildKit
                https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   68.1kB
Step 1/5 : FROM python:3.7
```

01:07:57

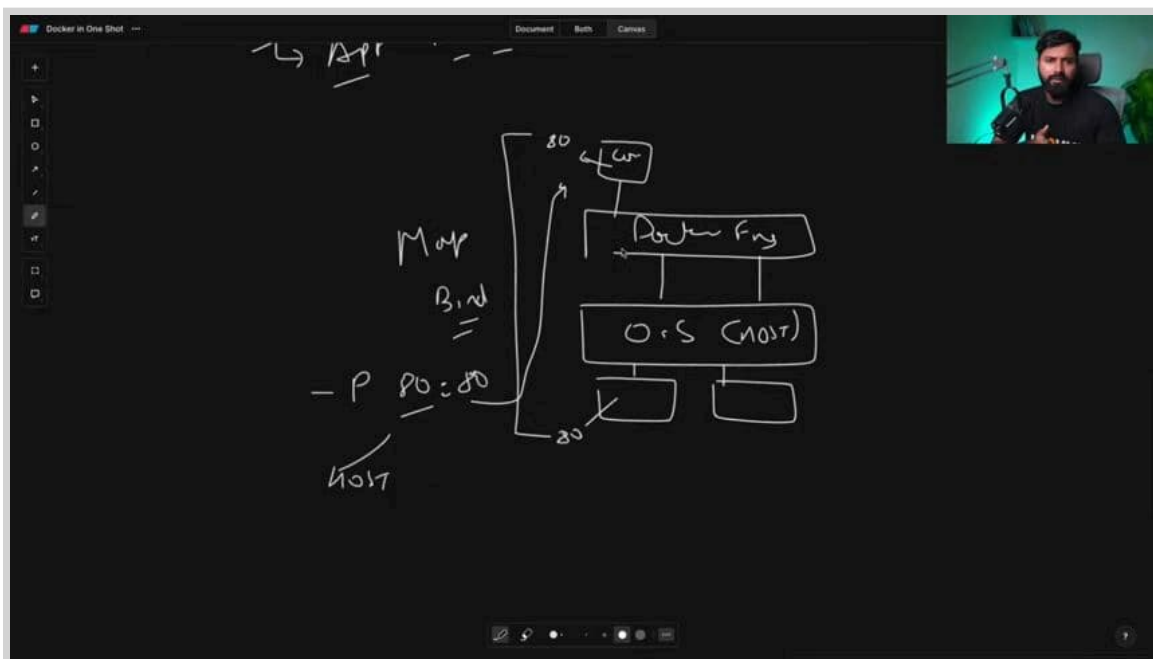


The screenshot shows a GitHub repository for 'flask-app-ecs' by user 'LondheShubham153'. The 'Dockerfile' is selected, showing the following code:

```
1 FROM python:3.7
2
3 RUN apt-get update -y
4 COPY ./ /app
5 WORKDIR /app
6 RUN pip install flask
7 ENTRYPOINT [ "python" ]
8 CMD [ "run.py" ]
```

Port mapping:-

[01:10:07](#)



[01:10:32](#)


```
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ docker run -d -p
p
bc228892034a65d8cddb45d2c1e1bd88418ba5b36daf081025fedc5c97f0897
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
PORTS         NAMES
bc228892034a   flask-app "python run.py"         6 seconds ago Up 5 secon
ds           0.0.0.0:80->80/tcp, :::80->80/tcp   serene_mestorf
2fcba592fbb2   mysql     "docker-entrypoint.s..." 27 minutes ago Up 27 minu
tes         3306/tcp, 33060/tcp              funny_turing
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$
```

==>To check container logs

docker logs <containerID>

01:13:15

```
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ docker logs bc228
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a product
. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:80
* Running on http://172.17.0.3:80
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 725-001-027
103.117.218.40 - - [06/Nov/2024 10:01:26] "GET / HTTP/1.1" 200 -
103.117.218.40 - - [06/Nov/2024 10:01:26] "GET /favicon.ico HTTP/1.1" 404 -
103.117.218.40 - - [06/Nov/2024 10:01:30] "GET / HTTP/1.1" 200 -
103.117.218.40 - - [06/Nov/2024 10:01:41] "GET /health HTTP/1.1" 200 -
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$
```

to see live logs jisse tumhara terminal or docker ka terminal
attach ho jaaye to chalana padata hai

==> docker attach <container ID>

01:14:20


```
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ docker attach bc2
103.117.218.40 - - [06/Nov/2024 10:03:01] "GET /profile HTTP/1.1"
103.117.218.40 - - [06/Nov/2024 10:03:02] "GET /profile HTTP/1.1"
103.117.218.40 - - [06/Nov/2024 10:03:03] "GET /profile HTTP/1.1"
103.117.218.40 - - [06/Nov/2024 10:03:05] "GET / HTTP/1.1" 200 -
103.117.218.40 - - [06/Nov/2024 10:03:11] "GET / HTTP/1.1" 200 -
```



aap ka terminal block ho gya hai container run karne se to
container ko stop kar do
==> docker stop <container ID>

01:14:58

```
ubuntu@ip-172-31-7-73:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
5              NAMES
fb293b16554f   mysql     "docker-entrypoint.s..." About a minute ago Up About
/tcp, 33060/tcp vigilant_napier
ubuntu@ip-172-31-7-73:~$ docker stop fb293b16554f
fb293b16554f
ubuntu@ip-172-31-7-73:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
2fcba592fbb2   mysql     "docker-entrypoint.s..." 14 seconds ago Up 13 seconds 3306/tcp, 33
060/tcp funny_turing
ubuntu@ip-172-31-7-73:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
bc228892034a   flask-app "python run.py"          4 minutes ago Up 4 minutes  0.0.0.0:80
->80/tcp, :::80->80/tcp serene_mestorf
2fcba592fbb2   mysql     "docker-entrypoint.s..." 31 minutes ago Up 31 minutes 3306/tcp,
33060/tcp funny_turing
ubuntu@ip-172-31-7-73:~$ docker stop bc228892034a
bc228892034a
ubuntu@ip-172-31-7-73:~$ docker stop bc228892034a
```



01:15:18

```

$ NAMES
fb293b16554f mysql "docker-entrypoint.s..." About a minute ago Up About
/tcp, 33060/tcp vigilant_napier
ubuntu@ip-172-31-7-73:~$ docker stop fb293b16554f
fb293b16554f
ubuntu@ip-172-31-7-73:~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
NAMES
2fcba592fbb2 mysql "docker-entrypoint.s..." 14 seconds ago Up 13 seconds 3306/tcp, 33
060/tcp funny_turing
ubuntu@ip-172-31-7-73:~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
NAMES
bc228892034a flask-app "python run.py" 4 minutes ago Up 4 minutes 0.0.0.0:80
->80/tcp, :::80->80/tcp serene_mestorf
2fcba592fbb2 mysql "docker-entrypoint.s..." 31 minutes ago Up 31 minutes 3306/tcp,
33060/tcp funny_turing
ubuntu@ip-172-31-7-73:~$ docker stop bc228892034a
bc228892034a
ubuntu@ip-172-31-7-73:~$ docker start bc228892034a
bc228892034a
ubuntu@ip-172-31-7-73:~$ docker stop bc228892034a

```

01:16:05

```

ubuntu@ip-172-31-7-73:~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS
NAMES
bc228892034a flask-app "python run.py" 5 minutes ago Up 16 sec
->80/tcp, :::80->80/tcp serene_mestorf
2fcba592fbb2 mysql "docker-entrypoint.s..." 32 minutes ago Up 32 minutes 3306/tcp,
33060/tcp funny_turing
ubuntu@ip-172-31-7-73:~$

```

=====

container ke andar jaake commands runkarni hai to

01:16:55

```
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED
PORTS         NAMES
bc228892034a   flask-app  "python run.py"         5 minutes ago
0.0.0.0:80->80/tcp, :::80->80/tcp  serene_mestorf
2fcba592fbb2   mysql     "docker-entrypoint.s..." 33 minutes ago  Up 33 minu
tes          3306/tcp, 33060/tcp        funny_turing
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ docker exec -it 2fcba592fbb2 ba
sh
bash-5.1#
```

docker exec -it <container ID> <shell name>

[01:17:07](#)

```
2fcba592fbb2   mysql     "docker-entrypoint.s..." 33 minutes ago
tes          3306/tcp, 33060/tcp        funny_turing
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ docker exec -it 2fcba592fbb2
sh
bash-5.1# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 9.1.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

[01:17:21](#)

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.00 sec)

mysql> craete database devops;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near 'cra
ete database devops' at line 1
mysql> 
```

[01:17:49](#)

```
Query OK, 1 row affected (0.01 sec)

mysql> show databases;
+-----+
| Database |
+-----+
| devops |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

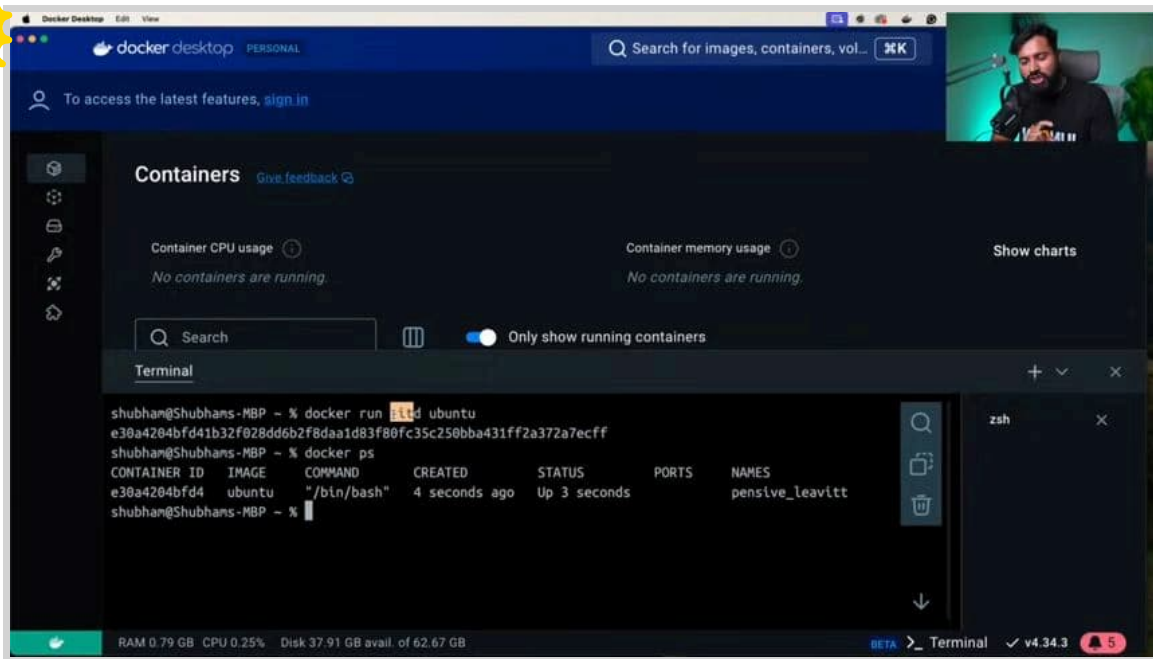
mysql> exit
Bye
bash-5.1# exit
exit
ubuntu@ip-172-31-7-73:~/projects/flask-app-ecs$ 
```

=====

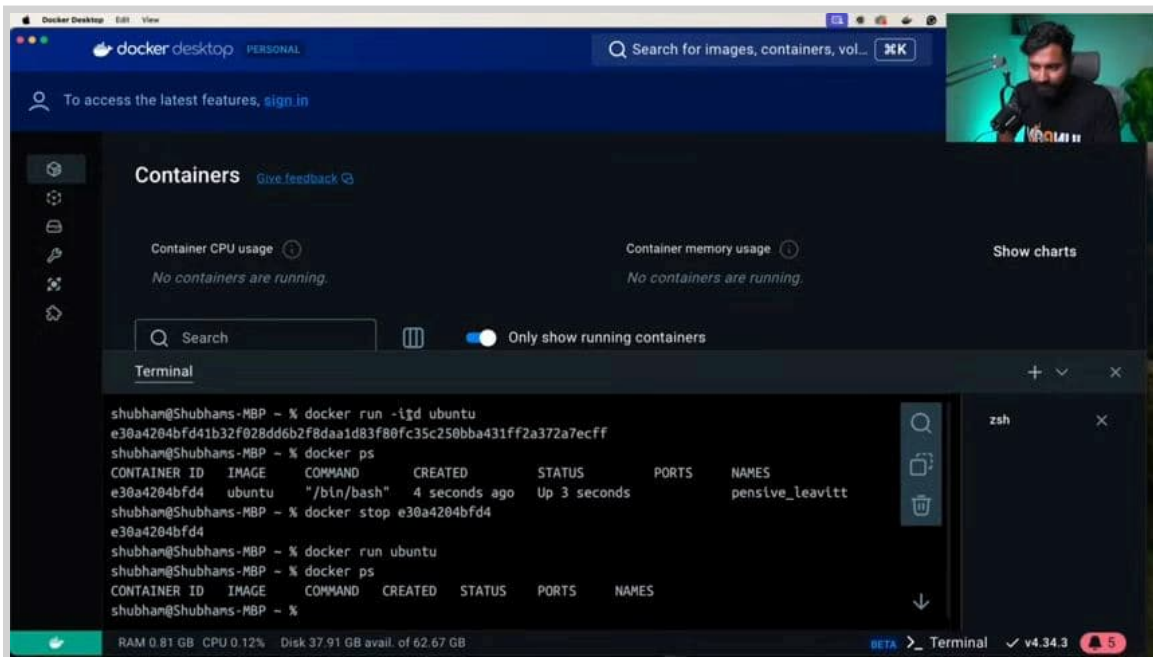
mujhe Mac OS me ubuntu chalana hai or uska interactive terminal hamesh dikhta rhe

jaise kai containers ka kaam sirf itna hota hia ki chal kar band ho jaana, agar aap chahte hai ki vo continuous li chalta rhe to "-it" lagana hoga docker cmmmand ke baad or "-d" detach mode me

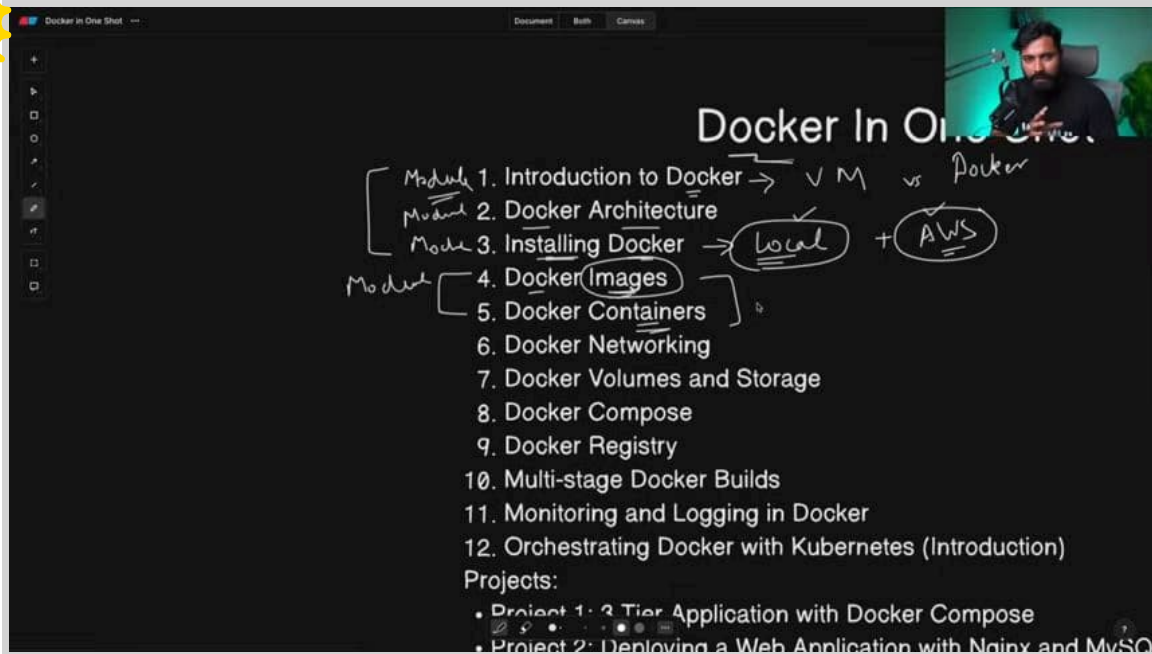
[01:19:32](#)



01:20:11

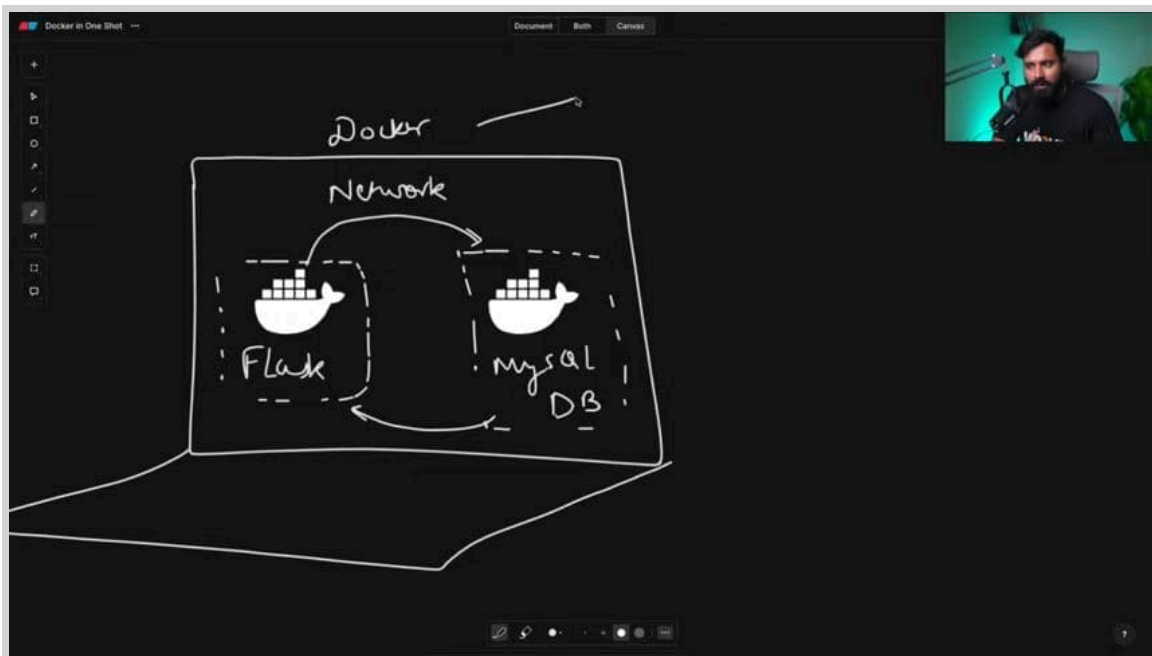


01:20:45

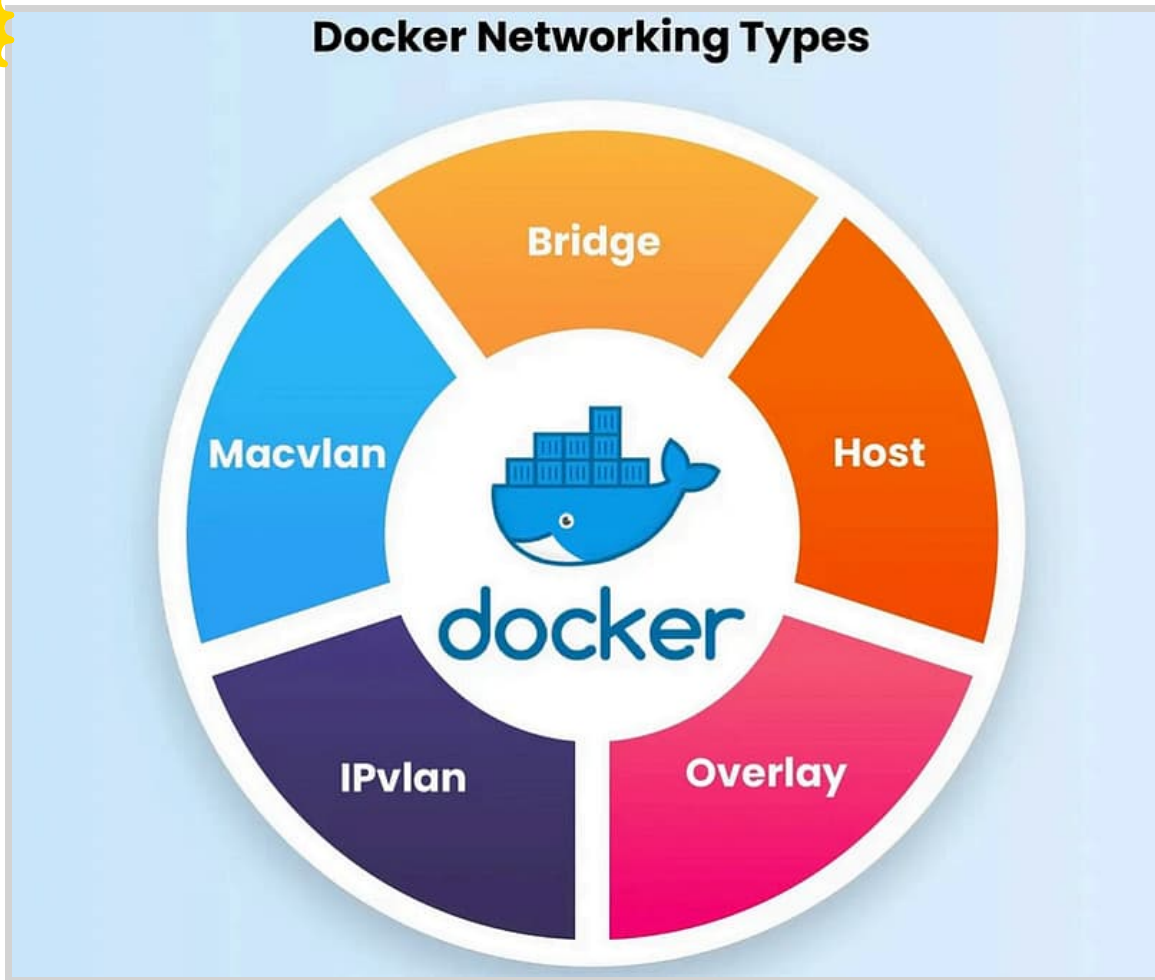


Docker Networking

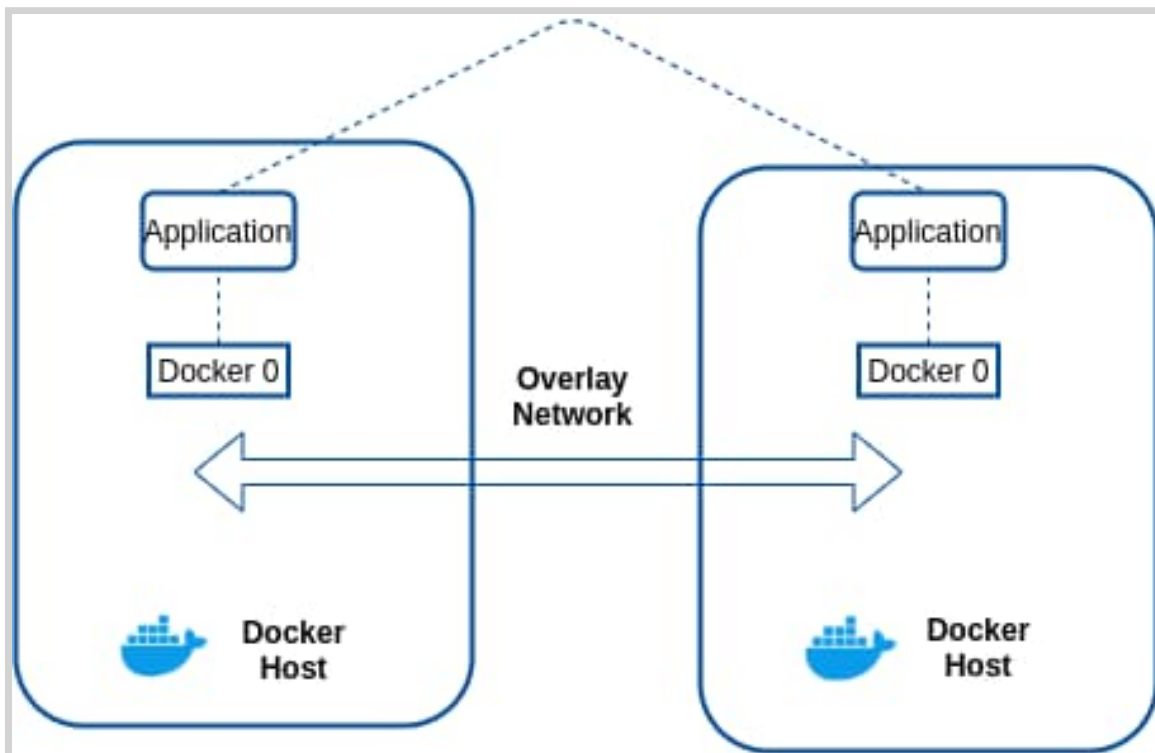
01:23:01



Docker Networking



 <https://www.docker.com/app/uploads/2022/12/networking-drivers-use-cases-3.png>



What is Docker Networking?

Docker Networking allows **containers to communicate** with:

- Other containers
- The host system
- External networks (internet)

Docker provides **virtual networks** so containers stay **isolated but connected** when required.

Why Docker Networking is Needed

- Container-to-container communication
- Application access from browser
- Microservices communication
- Isolation and security

Types of Docker Networks

1 Bridge Network (Default)

- Default network for containers
- Containers on same bridge can talk using **container name**
- Uses NAT

 Example:

```
docker network ls
docker run -d --name web nginx
docker run -d --name app --network bridge
myapp
```

 Most commonly used on **single host**

2 Host Network

- Container shares **host's network**
- No isolation between container and host
- Best performance

```
docker run --network host nginx
```

📌 Use when **low latency** is required

3 Overlay Network

- Used in **Docker Swarm**
- Enables container communication across **multiple hosts**

📌 Used for **microservices**

4 None Network

- No networking
- Fully isolated container

```
docker run --network none busybox
```

5 Macvlan Network

- Container gets its **own IP from LAN**
- Appears as physical device

📌 Used when app needs direct LAN access

Docker Network Commands

```
docker network ls
```

```
docker network inspect bridge
```

```
docker network create mynet
```

```
docker network connect mynet container1
```

```
docker network disconnect mynet container1
```

How Containers Communicate

- Same network → communicate using **container name**



- Different network → cannot communicate unless connected manually
- External access → **port mapping**

```
docker run -p 8080:80 nginx
```

Port Mapping (Interview Favorite)

Host Port : Container Port

```
-p 8080:80
```

Browser → `http://host_ip:8080`

Real-World Scenario Example

We deployed a web app and database in separate containers on a custom bridge network.

The web container accessed the database using the DB container name, improving security and isolation.

One-Line Interview Answer

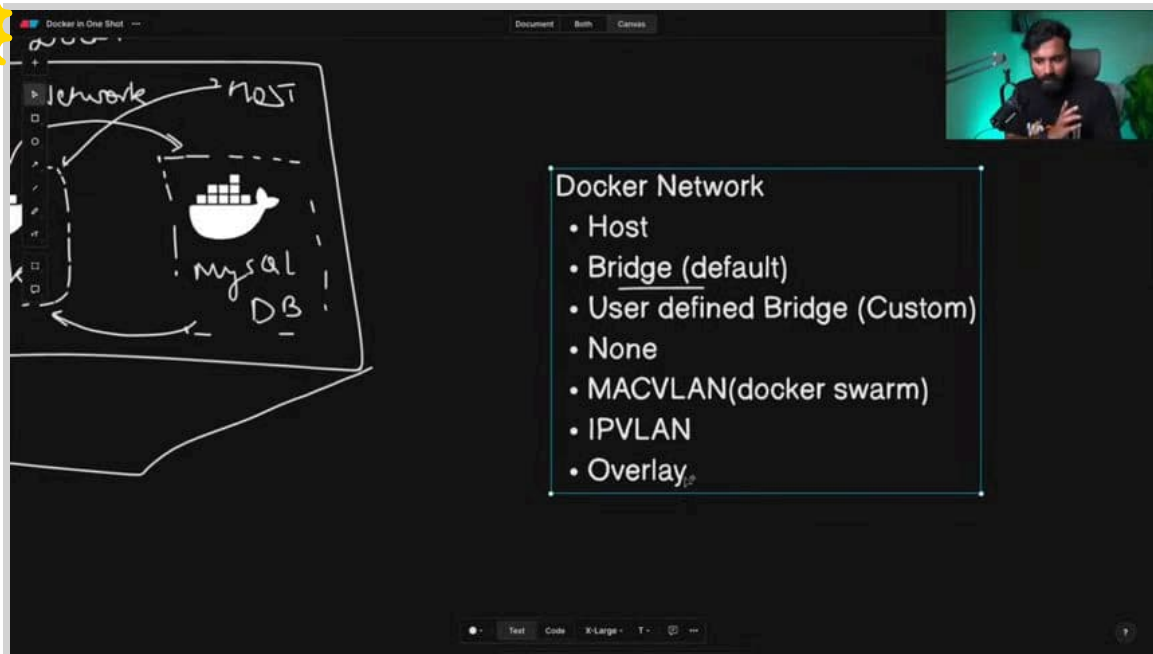
Docker Networking allows containers to communicate with each other and external systems using isolated virtual networks like bridge, host, and overlay.

Common Interview Troubleshooting Questions

- App not accessible → check **port mapping**
- Containers not communicating → check **network**
- DNS issue → use **container name**
- Production → prefer **custom bridge network**

==> docker swarm ka use aaj kal nhi karte uski jaga kubernetes ka use karte hai.

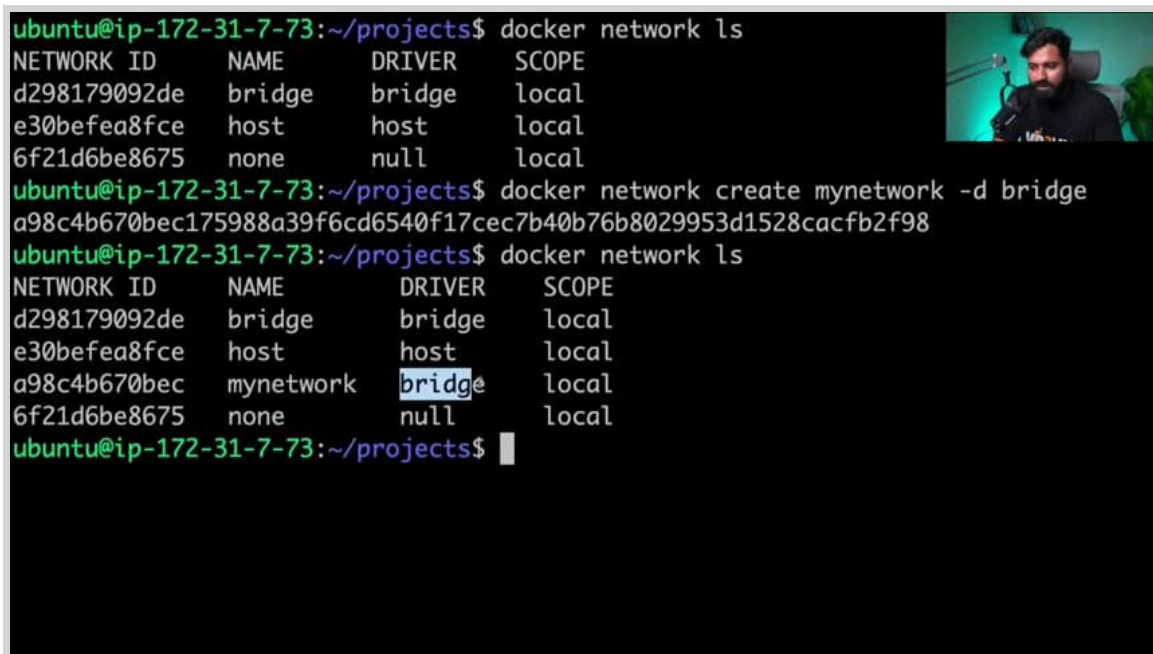
[01:25:41](#)



==>to create network

docker network create <network name> -d <which type of network you want to create>

[01:26:53](#)

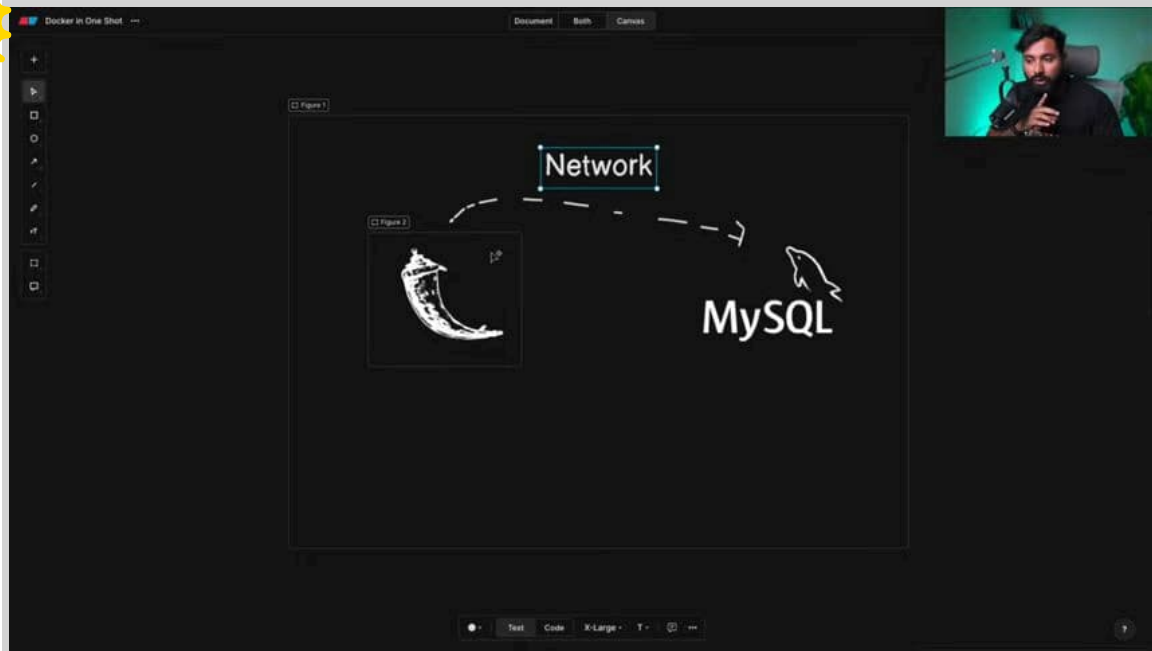


=====

2-tier project

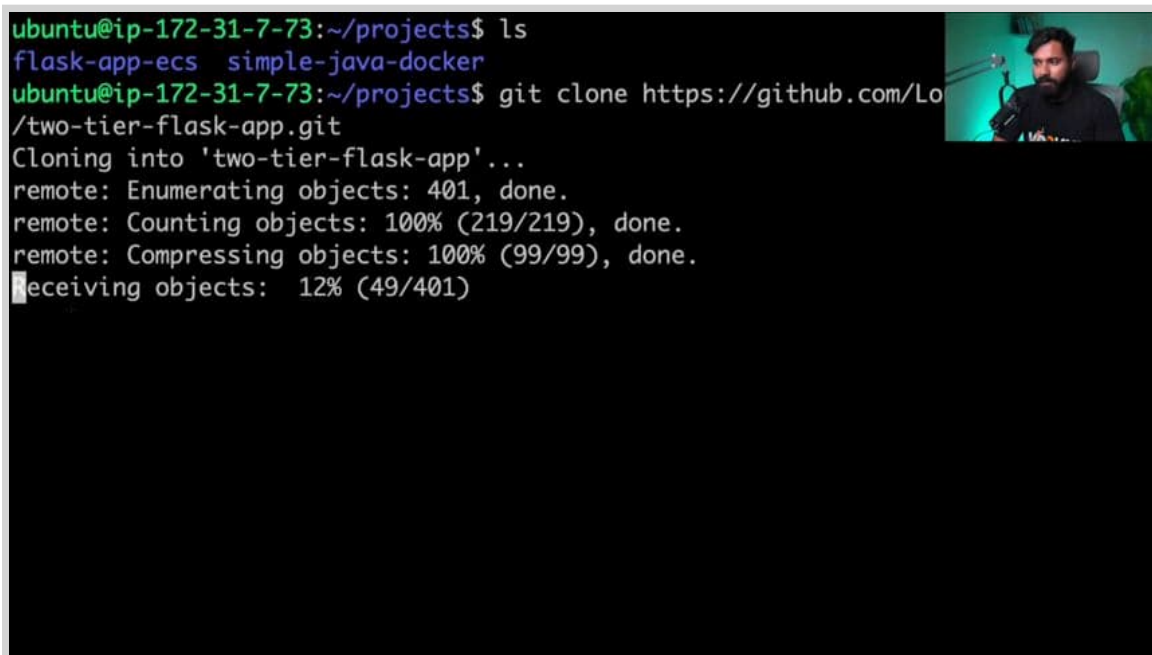
=====

[01:28:32](#)



flask ki application ko mysql database se connect karna hai
====> go to browser search 2-tier flask app clone it on local machine

01:29:10



01:29:27

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ ls
Dockerfile          Makefile  docker-compose.yml  message.sql
Dockerfile-multistage  README.md  eks-manifests       requirement
Jenkinsfile         app.py    k8s                 templates
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ cat app.py
```

[01:29:44](#)

```
app = Flask(__name__)

# Configure MySQL from environment variables
app.config['MYSQL_HOST'] = os.environ.get('MYSQL_HOST', 'localhost')
app.config['MYSQL_USER'] = os.environ.get('MYSQL_USER', 'default_user')
app.config['MYSQL_PASSWORD'] = os.environ.get('MYSQL_PASSWORD', 'default_password')
app.config['MYSQL_DB'] = os.environ.get('MYSQL_DB', 'default_db')

# Initialize MySQL
mysql = MySQL(app)

def init_db():
    with app.app_context():
        cur = mysql.connection.cursor()
        cur.execute('''
            CREATE TABLE IF NOT EXISTS messages (
                id INT AUTO_INCREMENT PRIMARY KEY,
```

yaha bas itna samjhna hai kya ye application kisi database ko connect karta hai to jawab hai haa karta hai
==> ye kuch environment variables hai jo aap ko paas karne hoge is flask app ko chalane ke liye
docker run -e isme jo "-e" hai usse environment variable paas hota hai

[01:30:39](#)


```
Use an official Python runtime as the base image
FROM python:3.9-slim

# Set the working directory in the container
WORKDIR /app

# install required packages for system
RUN apt-get update \
    && apt-get upgrade -y \
    && apt-get install -y gcc default-libmysqlclient-dev pkg-config \
    && rm -rf /var/lib/apt/lists/*

# Copy the requirements file into the container
COPY requirements.txt .

# Install app dependencies
RUN pip install mysqlclient
RUN pip install --no-cache-dir -r requirements.txt
"Dockerfile" 25L, 628B 1,1 Top
```

[01:31:37](#)

```
Use an official Python runtime as the base image
FROM python:3.9-slim

# Set the working directory in the container
WORKDIR /app

# install required packages for system
RUN apt-get update \
    && apt-get upgrade -y \
    && apt-get install -y gcc default-libmysqlclient-dev pkg-config \
    && rm -rf /var/lib/apt/lists/*

# Copy the requirements file into the container
COPY requirements.txt .

# Install app dependencies
RUN pip install mysqlclient
RUN pip install --no-cache-dir -r requirements.txt
"Dockerfile" 25L, 628B 1,1 Top
```

[01:32:46](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker build -t two-tier-backend .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.

Install the buildx component to build images with Buildx
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 216.6kB
Step 1/8 : FROM python:3.9-slim
```



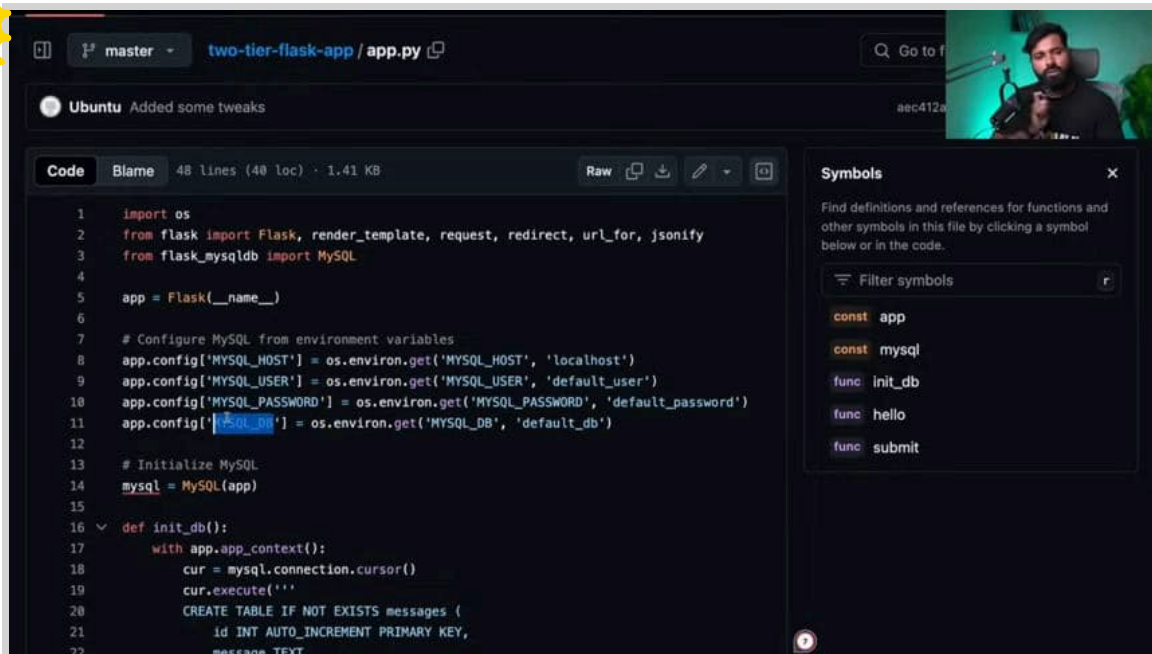
[01:33:41](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED
PORTS         NAMES
bc228892034a   flask-app  "python run.py"         23 minutes ago
0.0.0.0:80->80/tcp, :::80->80/tcp   serene_mestorf
2fcba592fbb2   mysql     "docker-entrypoint.s..." 50 minutes ago   Up 50 minutes
3306/tcp, 33060/tcp   funny_turing
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$
```



mujhe is container ko connect karne ke liye ye 4 variables chahiye or ye port 5000 par chal rha hai

[01:34:12](#)



```

1 import os
2 from flask import Flask, render_template, request, redirect, url_for, jsonify
3 from flask_mysql import MySQL
4
5 app = Flask(__name__)
6
7 # Configure MySQL from environment variables
8 app.config['MYSQL_HOST'] = os.environ.get('MYSQL_HOST', 'localhost')
9 app.config['MYSQL_USER'] = os.environ.get('MYSQL_USER', 'default_user')
10 app.config['MYSQL_PASSWORD'] = os.environ.get('MYSQL_PASSWORD', 'default_password')
11 app.config['MYSQL_DB'] = os.environ.get('MYSQL_DB', 'default_db')
12
13 # Initialize MySQL
14 mysql = MySQL(app)
15
16 def init_db():
17     with app.app_context():
18         cur = mysql.connection.cursor()
19         cur.execute('')
20         CREATE TABLE IF NOT EXISTS messages (
21             id INT AUTO_INCREMENT PRIMARY KEY,
22             message TEXT

```

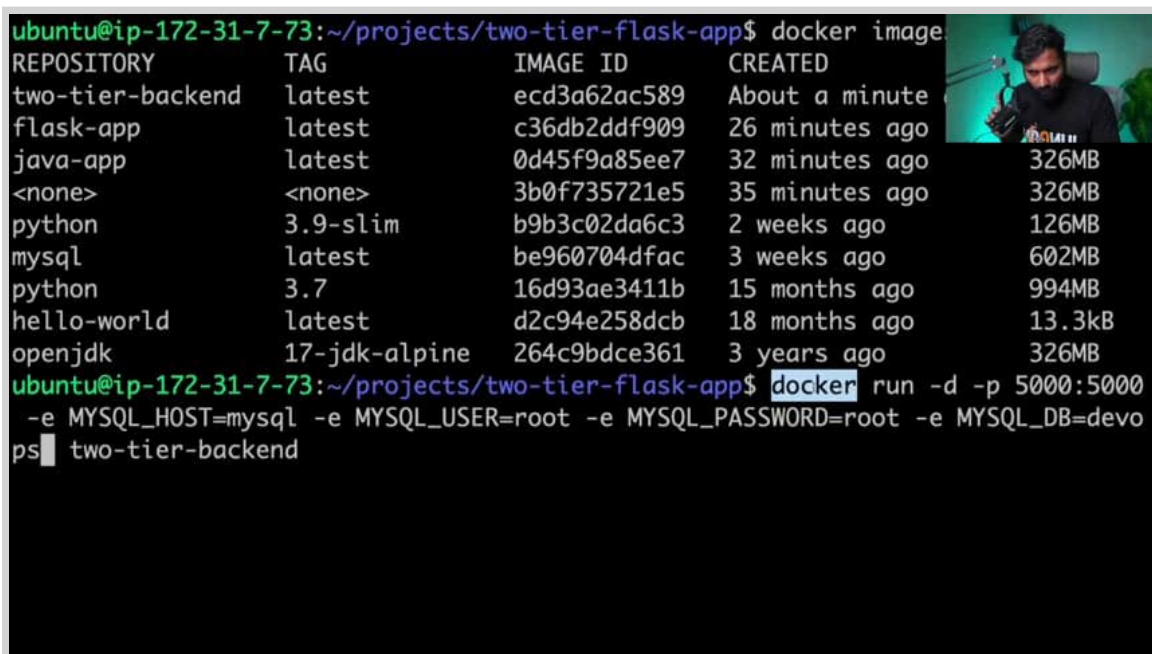
Symbols

Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.

Filter symbols

- const app
- const mysql
- func init_db
- func hello
- func submit

01:36:15



```

ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker image
REPOSITORY          TAG          IMAGE ID          CREATED
two-tier-backend    latest      ecd3a62ac589     About a minute
flask-app            latest      c36db2ddf909     26 minutes ago
java-app            latest      0d45f9a85ee7     32 minutes ago
<none>              <none>      3b0f735721e5     35 minutes ago
python              3.9-slim    b9b3c02da6c3     2 weeks ago
mysql               latest      be960704dfac     3 weeks ago
python              3.7         16d93ae3411b     15 months ago
hello-world         latest      d2c94e258dcb     18 months ago
openjdk             17-jdk-alpine 264c9bdce361     3 years ago
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker run -d -p 5000:5000
-e MYSQL_HOST=mysql -e MYSQL_USER=root -e MYSQL_PASSWORD=root -e MYSQL_DB=devo
ps two-tier-backend

```

01:36:57

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED
PORTS         NAMES
bc228892034a   flask-app  "python run.py"         26 minutes ago
0.0.0.0:80->80/tcp, :::80->80/tcp   serene_mestorf
2fcba592fbb2   mysql     "docker-entrypoint.s..." 53 minutes ago   Up 53 minu
3306/tcp, 33060/tcp               funny_turing
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$
```

container dikhai nhi de rha matlab container exit ho gaya hai
agar exit yaa inactive container ko dekhana hai to
==> docker ps -a se all container show ho jaayege

[01:37:20](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  STATUS      PORTS
582dd92f8729   two-tier-backend:latest  "python app.py"    Exited (1) 28 seconds ago
tic_ganguly
bc228892034a   flask-app  "python run.py"         Up 21 minutes    0.0.0.0:80->80/tcp, :::80->80/tcp   seren
e_mestorf
f6f88e888d67   java-app   "java Main"            Exited (0) 35 minutes ago
ge_hofstadter
d7d23e5f5443   3b0f735721e5  "java Main"            Exited (0) 36 minutes ago
_wing
53ed9e356b1a   3b0f735721e5  "java Main"            Exited (0) 37 minutes ago
easley
2fcba592fbb2   mysql     "docker-entrypoint.s..." 53 minutes ago
```

status exited hai iska reason pata lagane ke liye iske logs check
karege
docker logs <container ID>

[01:37:35](#)


```
Traceback (most recent call last):
  File "/app/app.py", line 46, in <module>
    init_db()
  File "/app/app.py", line 18, in init_db
    cur = mysql.connection.cursor()
  File "/usr/local/lib/python3.9/site-packages/flask_mysql_db/__init__.py", line
94, in connection
    ctx.mysql_db = self.connect
  File "/usr/local/lib/python3.9/site-packages/flask_mysql_db/__init__.py", line
81, in connect
    return MySQLdb.connect(**kwargs)
  File "/usr/local/lib/python3.9/site-packages/MySQLdb/__init__.py", line 121,
in Connect
    return Connection(*args, **kwargs)
  File "/usr/local/lib/python3.9/site-packages/MySQLdb/connections.py", line 19
5, in __init__
    super().__init__(*args, **kwargs2)
MySQLdb.OperationalError: (2005, "Unknown server host 'mysql' (-2)")
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$
```

unknown server error ko pata nhi ahi ki msq kon hai kyoki abhi
vo same network par nhi hai iseke liye
mysql ka container ka naam funny_turing hai
mysql_host ki jaga hai container ka naam dekar dete hai

=====

ab network banate hai or undono ko same network par laate hai
container ko stop or remove kar diya

01:39:21

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker ps
CONTAINER ID   IMAGE      COMMAND                  NAMES          CREATED
PORTS
bc228892034a   flask-app  "python run.py"         serene_mestorf  28 minutes ago
0.0.0.0:80->80/tcp, :::80->80/tcp
2fcba592fbb2   mysql     "docker-entrypoint.s..."  funny_turing    55 minutes ago Up 55 minu
3306/tcp, 33060/tcp
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker stop 2fcba592fbb2 &
& docker rm 2fcba592fbb2
```

==> create network

[01:40:16](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker network create two-tier -d bridge  
c56883e3a3b45bec8f6f5bf0bdae580f01ad125eb0a20465e1042d81c9847a13  
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$
```



ab network dono containers ko dedete hai

[01:41:43](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker network create two-tier -d bridge  
c56883e3a3b45bec8f6f5bf0bdae580f01ad125eb0a20465e1042d81c9847a13  
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker run -d --name mysql --network two-tier -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=devops mysql
```



ab ye same network hum dege apne flask ke container ko

[01:42:45](#)


```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker network create two-tier -d bridge
c56883e3a3b45bec8f6f5bf0bdae580f01ad125eb0a20465e1042d81c9847a13
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker run -d --name mysql --network two-tier -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=devops mysql
ec0ab4df88dd54393b079eaae51adbbbbed0735cf4678bbcf5376d96f354612f9
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker run -d --network two-tier -e MYSQL_HOST=mysql -e MYSQL_USER=root -e MYSQL_PASSWORD=root -e MYSQL_DB=devops two-tier-backend:latest
0311d33c31d936f1c0140810c65ef42d04387a7e0dc834872c81346dcfc1dc4a
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$
```

[01:42:52](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	NAMES
0311d33c31d9	two-tier-backend:latest	"python app.py"	5 seconds ago	magical_grothendieck
ec0ab4df88dd	mysql	"docker-entrypoint.s..."	About a minute ago	
bc228892034a	flask-app	"python run.py"	Up 27 minutes	
mestorf				

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$
```

[01:43:07](#)

```

Up 27 minutes      0.0.0.0:80->80/tcp, :::80->80/tcp      serene_
mestorf
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker logs 0311d33c31d9
* Serving Flask app 'app' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.19.0.3:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 138-272-591
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$

```

network ko inspect karke pata laga skte hai is network me kitne containers hai

[01:43:40](#)

```

ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker network
NETWORK ID      NAME      DRIVER      SCOPE
d298179092de    bridge    bridge      local
e30bfe8a8f8e    host      host        local
a98c4b670bec    mynetwork bridge      local
6f21d6be8675    none      null        local
c56883e3a3b4    two-tier  bridge      local
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker network inspect two-tier

```

[01:43:47](#)

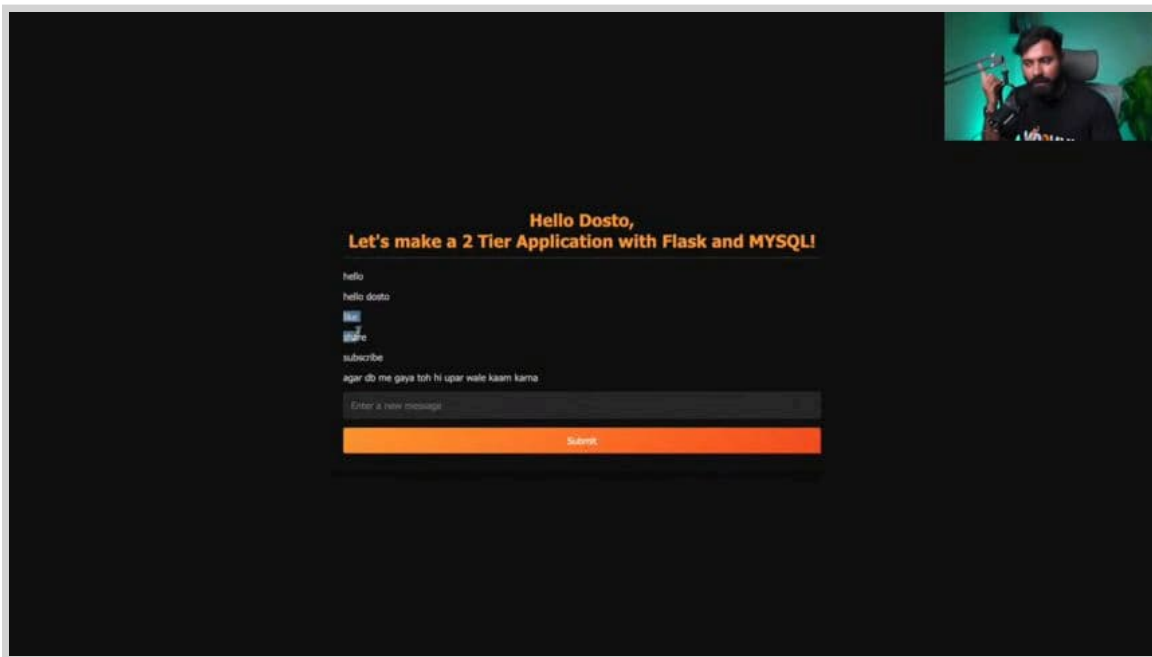
```

    }
  ],
  "Internal": false,
  "Attachable": false,
  "Ingress": false,
  "ConfigFrom": {
    "Network": ""
  },
  "ConfigOnly": false,
  "Containers": {
    "0311d33c31d936f1c0140810c65ef42d04387a7e0dc834872c81346dcfc1dc4a": {
      "Name": "magical_grothendieck",
      "EndpointID": "f1ea3ef3403605da9eb6f9f883a3180f7ec3dbad4e4fcf45747c63e558374443",
      "MacAddress": "02:42:ac:13:00:03",
      "IPv4Address": "172.19.0.3/16",
      "IPv6Address": ""
    }
  }
}

```

firewall me port 5000 ko active kar dena hai

[01:45:17](#)



ab me msql ke container me jaake check karuga ki apna data show ho rha ahi kya

[01:46:00](#)

```
dd bash
bash-5.1# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 17
Server version: 9.1.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use database devops;
ERROR 1049 (42000): Unknown database 'database'
mysql> show databases
-> ;
```

01:46:15

```
mysql> show databases
-> ;
+-----+
| Database |
+-----+
| devops   |
| information_schema |
| mysql    |
| performance_schema |
| sys      |
+-----+
5 rows in set (0.00 sec)

mysql> use devops;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from message
```

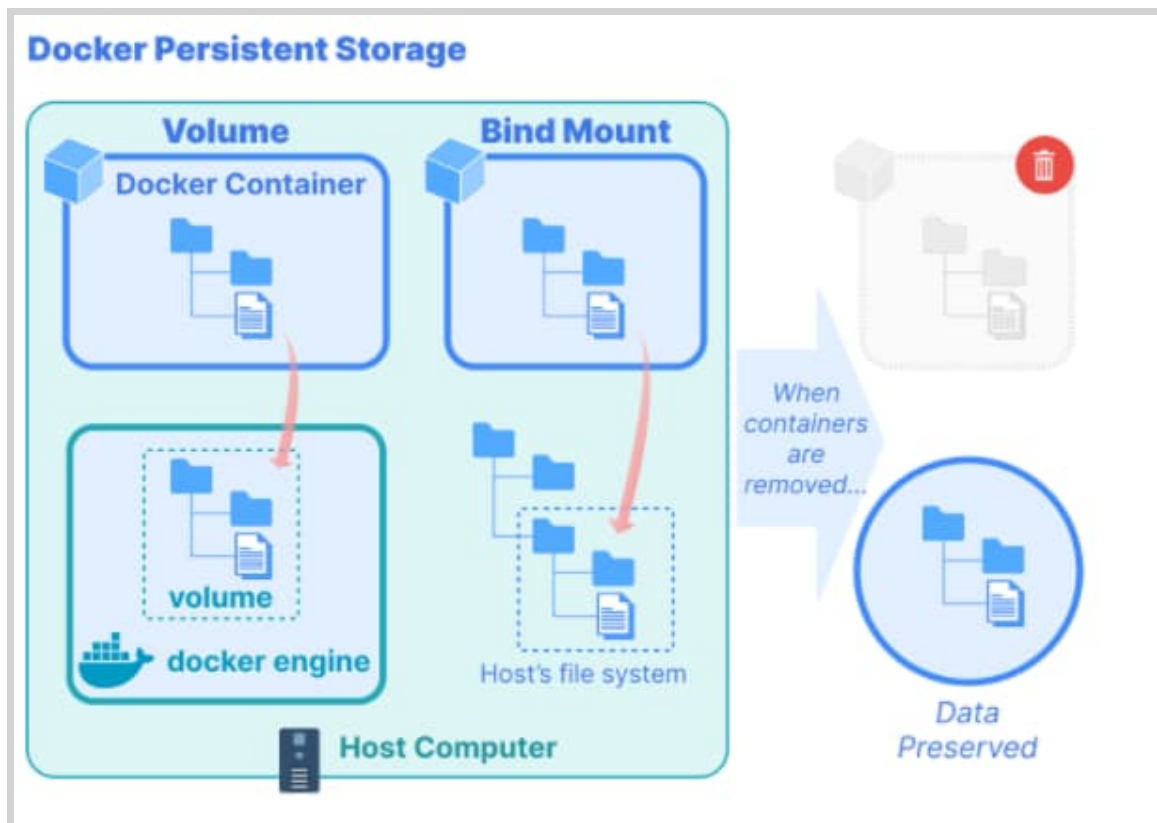
=====
==> agar me apne mysql ke container ko restart karu to mujhe database dikhai dega
lekin agar conatiner ko remove kar du to saare data delete ho jaayega

Docker Volumes and Sotrage:-

Docker Volumes & Storage

 <https://docker-docs.uclv.cu/storage/images/types-of-mounts-bind.png>

 <https://docs.docker.com/engine/storage/images/volumes-shared-storage.webp>



Docker Storage ki Problem (Why Volumes?)

By default:

- Container ke andar ka data **ephemeral** hota hai
- Container delete → **data delete**

👉 Databases, logs, uploads ke liye ye **acceptable nahi**.

Is problem ko solve karne ke liye Docker **Volumes & Mounts** provide karta hai.

Docker Storage Types (VERY IMPORTANT)

Docker me mainly 3 types hote hain:

1 Docker Volumes (Best & Recommended)

What is Docker Volume?

Docker Volume ek **Docker-managed persistent storage** hota hai jo container lifecycle se **independent** hota hai.

- Data container delete hone ke baad bhi safe
- Docker khud manage karta hai
- Production ke liye best

Example

```
docker volume create dbdata
```

```
docker run -d \  
-v dbdata:/var/lib/mysql \  
mysql
```

Features

- Secure
 - Easy backup & restore
 - Shareable between containers
-

2 Bind Mounts

What is Bind Mount?

Host system ka koi **specific directory** container ke andar mount karna.

Example


```
docker run -d \
-v /opt/appdata:/app/data \
myapp
```

Use case

- Development
- Live code changes

Drawback

- Less portable
- Host dependency
- Security risk

3 tmpfs Mounts

What is tmpfs?

- Data RAM me store hota hai
- Container stop → data gone

Use case

- Secrets
- Temporary data

Volume vs Bind Mount (Interview Table)

Feature	Volume	Bind Mount	Managed by
Docker	Yes	No	Persistent
Yes	Yes	Yes	Portability
High	Low	Security	High
Lower	Best for	Production	Development

How Volumes Work Internally

- Stored in:

`/var/lib/docker/volumes/`

- Volume container se attach hota hai
- Multiple containers ek volume share kar sakte hain

Docker Volume Commands

```
docker volume ls
docker volume create myvol
docker volume inspect myvol
docker volume rm myvol
```

Real-World Example (Interview GOLD)

We used Docker volumes to persist MySQL database data.

Even when containers were recreated during deployments, the database data remained intact.

Backup & Restore of Docker Volume

Backup

```
docker run --rm \
-v myvol:/data \
-v $(pwd):/backup \
ubuntu tar cvf /backup/backup.tar /data
```

Restore

```
docker run --rm \
-v myvol:/data \
-v $(pwd):/backup \
ubuntu tar xvf /backup/backup.tar -C /
```

INTERVIEW QUESTIONS (VERY IMPORTANT)

Basic

1. What is Docker Volume?
2. Why do we need Docker volumes?
3. Where are Docker volumes stored?
4. What happens to volume when container is deleted?

Intermediate

1. Difference between volume and bind mount?
2. Can multiple containers use the same volume?
3. How do you take backup of Docker volume?
4. How do you persist database data in Docker?

Advanced / Scenario-Based

1. Container deleted accidentally. Database data kaise safe raha?
2. Production me bind mount kyun avoid karte hain?
3. Volume ka data migrate kaise karoge?
4. Docker volume corrupt ho gaya, kya steps loge?

One-Line Interview Answer

Docker volumes provide persistent storage that is independent of container lifecycle and are used to store critical data like databases and logs.

Interviewer Impress Tip

Bolna:

“For production, I always prefer Docker volumes over bind mounts because they are secure, portable, and Docker-managed.”

container ke data ko bind kar lete hai host ke saath vaha ek filesystem yaa folder create karke. or is concept ko volume kahte hai

==> docker volume ls

[01:51:33](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker volum
DRIVER      VOLUME NAME
local       9a5dfec3cf1efc561e715c8477350b047aeb8c6ce227501c43e1a4c
local       99c9be0ca1939663d61d846c774fcc7ee0c5c37893cda58eab2a3a1
local       1193ce257023e8b0ca80af8571403ca335206107302190001b5fb02f751f7fc7
local       feb89388a41f797db3adc3b4b165eb00c97e5f01d90659f11f20c4f7e9bcce96
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$
```

==> want to create your own volume

docker volume create <volume name>

[01:51:58](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker volume create mysql
mysql-data
mysql-data
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$
```

[01:52:14](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker volume create mysql-data
mysql-data
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker inspect mysql-data
[
  {
    "CreatedAt": "2024-11-06T12:00:14Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mysql-data/_data",
    "Name": "mysql-data",
    "Options": null,
    "Scope": "local"
  }
]
```

ab ye volume by default konsi path par rhe gi ise pata kar skte hai
docker inspect <volume name>

==> mujhe apne container ko is volume(path) ke saath kaise
bind karuga

to this first stop and rm mysql container

and add a small addition in the command to run container of
mysql

[01:54:08](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker ps
CONTAINER ID   IMAGE          PORTS          COMMAND
STATUS        PORTS
438e3b88d0b9   mysql         3306/tcp, 33060/tcp  "docker-entrypoint.s..."
Up 3 minutes
0311d33c31d9   two-tier-backend:latest  "python app.py"
Up 2 minutes   0.0.0.0:5000->5000/tcp, :::5000->5000/tcp
ago
bc228892034a   flask-app     "python run.py"
Up 2 hours    0.0.0.0:80->80/tcp, :::80->80/tcp
2 hours ago
f
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker stop 438e3b88d0b9 &
& docker rm 438e3b88d0b9
438e3b88d0b9
438e3b88d0b9
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker run -d --name mysql
--network two-tier -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=root -e
MYSQL_DATABASE=devops mysql
```

==> now restart the flask application container

01:54:28

```
438e3b88d0b9  mysql "docker-entrypoint.s..."
Up 3 minutes 3306/tcp, 33060/tcp
0311d33c31d9  two-tier-backend:latest "python app.py"
ago Up 2 minutes 0.0.0.0:5000->5000/tcp, :::5000->5000/tcp
endieck
bc228892034a  flask-app "python run.py" 2 hours ago
Up 2 hours 0.0.0.0:80->80/tcp, :::80->80/tcp serene_mestor
f
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker stop 438e3b88d0b9 &
& docker rm 438e3b88d0b9
438e3b88d0b9
438e3b88d0b9
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker run -d --name mysql
--network two-tier -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=root -e
MYSQL_DATABASE=devops mysql
3bd80a6bb19c215e7972b033dcf5f3b2d8dd7e9c099c1933302ddf7bfe0c4d52
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker restart 0311d33c31d
9
```

aap ke container ke mysql ka data /var/lib/mysql ke andar stor
rhta hai

01:56:03

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker volume
l-data
[
  {
    "CreatedAt": "2024-11-06T12:00:14Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mysql-data/_data",
    "Name": "mysql-data",
    "Options": null,
    "Scope": "local"
  }
]
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ sudo su
root@ip-172-31-7-73:/home/ubuntu/projects/two-tier-flask-app#
```

01:56:08


```

    "Options": null,
    "Scope": "local"
  }
]
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ sudo su
root@ip-172-31-7-73:/home/ubuntu/projects/two-tier-flask-app# cd /var/lib/docke
r/volumes/mysql-data/_data
root@ip-172-31-7-73:/var/lib/docker/volumes/mysql-data/_data# ls
'#ib_16384_0.dblwr'  client-cert.pem      performance_schema
'#ib_16384_1.dblwr'  client-key.pem       private_key.pem
'#innodb_redo'      devops              public_key.pem
'#innodb_temp'      ib_buffer_pool       server-cert.pem
auto.cnf            ibdata1             server-key.pem
binlog.000001       ibtmp1              sys
binlog.000002       mysql               undo_001
binlog.index        mysql.ibd            undo_002
ca-key.pem          mysql.sock
ca.pem             mysql_upgrade_history
root@ip-172-31-7-73:/var/lib/docker/volumes/mysql-data/_data#

```

ab check karege apne mysql containerko delet karke

[01:56:13](#)

```

}
]
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ sudo su
root@ip-172-31-7-73:/home/ubuntu/projects/two-tier-flask-app# cd /var/lib/docke
r/volumes/mysql-data/_data
root@ip-172-31-7-73:/var/lib/docker/volumes/mysql-data/_data# ls
'#ib_16384_0.dblwr'  client-cert.pem      performance_schema
'#ib_16384_1.dblwr'  client-key.pem       private_key.pem
'#innodb_redo'      devops              public_key.pem
'#innodb_temp'      ib_buffer_pool       server-cert.pem
auto.cnf            ibdata1             server-key.pem
binlog.000001       ibtmp1              sys
binlog.000002       mysql               undo_001
binlog.index        mysql.ibd            undo_002
ca-key.pem          mysql.sock
ca.pem             mysql_upgrade_history
root@ip-172-31-7-73:/var/lib/docker/volumes/mysql-data/_data# exit
exit
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$

```

check kiya apna data safe hai

=====

Second method docker volume ke data persist karane ka

[01:57:36](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ cd
ubuntu@ip-172-31-7-73:~$ mkdir volumes
ubuntu@ip-172-31-7-73:~$ ls
projects  volumes
ubuntu@ip-172-31-7-73:~$ cd volumes/
ubuntu@ip-172-31-7-73:~/volumes$ mkdir mysql
ubuntu@ip-172-31-7-73:~/volumes$ ;s
bash: syntax error near unexpected token `;'
ubuntu@ip-172-31-7-73:~/volumes$ ls
mysql
ubuntu@ip-172-31-7-73:~/volumes$ pwd
/home/ubuntu/volumes
ubuntu@ip-172-31-7-73:~/volumes$ cd mysql/
ubuntu@ip-172-31-7-73:~/volumes/mysql$ pwd
/home/ubuntu/volumes/mysql
ubuntu@ip-172-31-7-73:~/volumes/mysql$
```



[01:57:41](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ cd
ubuntu@ip-172-31-7-73:~$ mkdir volumes
ubuntu@ip-172-31-7-73:~$ ls
projects  volumes
ubuntu@ip-172-31-7-73:~$ cd volumes/
ubuntu@ip-172-31-7-73:~/volumes$ mkdir mysql
ubuntu@ip-172-31-7-73:~/volumes$ ;s
bash: syntax error near unexpected token `;'
ubuntu@ip-172-31-7-73:~/volumes$ ls
mysql
ubuntu@ip-172-31-7-73:~/volumes$ pwd
/home/ubuntu/volumes
ubuntu@ip-172-31-7-73:~/volumes$ cd mysql/
ubuntu@ip-172-31-7-73:~/volumes/mysql$ pwd
/home/ubuntu/volumes/mysql
ubuntu@ip-172-31-7-73:~/volumes/mysql$ cd
ubuntu@ip-172-31-7-73:~$ cd
```



[01:58:22](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker ps
CONTAINER ID   IMAGE                                COMMAND
STATUS        PORTS
5c3bee59270e   mysql                              "docker-entrypoint.s..."
Up About a minute   3306/tcp, 33060/tcp
0311d33c31d9   two-tier-backend:latest           "python app.py"
Up About a minute   0.0.0.0:5000->5000/tcp, :::5000->5000/tcp
_grothendieck
bc228892034a   flask-app                         "python run.py"
Up 2 hours         0.0.0.0:80->80/tcp, :::80->80/tcp
mestorf
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker stop mysql && docke
r rm mysql
```

[01:58:53](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker ps
CONTAINER ID   IMAGE                                COMMAND
STATUS        PORTS
5c3bee59270e   mysql                              "docker-entrypoint.s..."
Up About a minute   3306/tcp, 33060/tcp
0311d33c31d9   two-tier-backend:latest           "python app.py"
Up About a minute   0.0.0.0:5000->5000/tcp, :::5000->5000/tcp
_grothendieck
bc228892034a   flask-app                         "python run.py"
Up 2 hours         0.0.0.0:80->80/tcp, :::80->80/tcp
mestorf
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker stop mysql && docke
r rm mysql
mysql
mysql
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker run -d --name mysql
--network two-tier -v /home/ubuntu/volumes/mysql:/var/lib/mysql -e MYSQL_ROOT_
PASSWORD=root -e MYSQL_DATABASE=devops mysql
```

[01:59:05](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ cd
ubuntu@ip-172-31-7-73:~$ cd volumes/
ubuntu@ip-172-31-7-73:~/volumes$ ls
mysql
ubuntu@ip-172-31-7-73:~/volumes$ cd mysql/
ubuntu@ip-172-31-7-73:~/volumes/mysql$ ls
'#ib_16384_0.dblwr'  client-cert.pem      performance_schema
'#ib_16384_1.dblwr'  client-key.pem       private_key.pem
'#innodb_redo'       ib_buffer_pool       public_key.pem
'#innodb_temp'       ibdata1              server-cert.pem
auto.cnf             ibtmp1               server-key.pem
binlog.000001        mysql                 sys
binlog.index         mysql.ibd             undo_001
ca-key.pem           mysql.sock            undo_002
ca.pem               mysql_upgrade_history
ubuntu@ip-172-31-7-73:~/volumes/mysql$
```

Docker Compose

abhi me bahut saara manual kaam kar rha hu to aap ko aapna kaam automate karne ke liye Docker Compose ki jarurat padegi

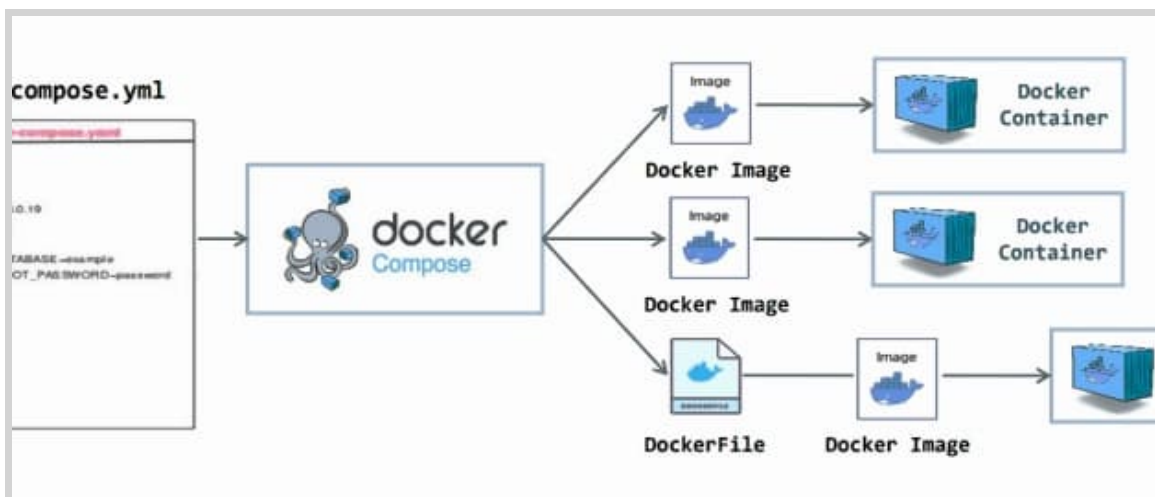
Docker Compose

 <https://docs.docker.com/compose/images/compose-application.webp>


```

1  version: '3.3'
2
3  services:
4    db:
5      image: mysql:5.7
6      volumes:
7        - db_data:C:\Users\User\Desktop\dcompose
8      restart: always
9      environment:
10       MYSQL_ROOT_PASSWORD: rootwordpress
11       MYSQL_DATABASE: wordpress
12       MYSQL_USER: wordpress
13       MYSQL_PASSWORD: wordpress
14
15     wordpress:
16       depends_on:
17         - db
18       image: wordpress:latest
19       ports:
20         - "8000:80"
21       restart: always
22       environment:
23         WORDPRESS_DB_HOST: db:3306
24         WORDPRESS_DB_USER: wordpress
25         WORDPRESS_DB_PASSWORD: wordpress
26   volumes:
27     db_data:

```



What is Docker Compose?

Docker Compose is a tool used to define and run multi-container applications using a single configuration file called `docker-compose.yml`.



With one command, we can:

- Start multiple containers
- Create networks
- Create volumes
- Manage dependencies



One file + one command = complete application stack

Why Docker Compose is Used

- Run **multiple services together** (web + db + cache)
- Easy setup for **development & testing**
- No need to run multiple docker run commands
- Clean & repeatable deployments

docker-compose.yml (Basic Example)

```
version: "3.8"
```

```
services:
```

```
  web:
```

```
    image: nginx
```

```
    ports:
```

```
      - "8080:80"
```

```
    depends_on:
```

```
      - app
```

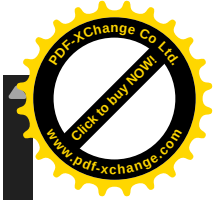
```
  app:
```

```
    image: myapp:v1
```

```
  db:
```

```
    image: mysql:8
```

```
    environment:
```

```
MYSQL_ROOT_PASSWORD: root
volumes:
  - dbdata:/var/lib/mysql
```

```
volumes:
  dbdata:
```

Important Sections in docker-compose.yml

- Defines Compose file format

services

- Each service = one container

image / build

- image → pull from registry
- build → build from Dockerfile

ports

- Port mapping

```
ports:
  - "8080:80"
```

volumes

- Persistent storage

```
volumes:
  - dbdata:/var/lib/mysql
```

networks

- Custom network for communication
-



depends_on

- Controls startup order

Docker Compose Commands (Interview Favorite)

```
docker-compose up
docker-compose up -d
docker-compose down
docker-compose ps
docker-compose logs
docker-compose build
docker-compose restart
```

How Networking Works in Docker Compose

- Docker Compose creates a **default bridge network**
- Containers communicate using **service name**

```
app → db (using hostname "db")
```

No IP needed 👍

Real-World Example (Interview GOLD)

We used Docker Compose to deploy a web application with Nginx, backend service, and MySQL.

All services ran on a single custom network, and database data was persisted using Docker volumes.

Docker Compose vs Dockerfile

Dockerfile Docker Compose Builds image Runs containers Single container Multi-container Image creation App orchestration

Production Note (Important)

- Docker Compose → **Dev / Test / Small setups**
- Production orchestration → **Kubernetes / Docker Swarm**

INTERVIEW QUESTIONS (VERY IMPORTANT)

Basic

1. What is Docker Compose?
2. What is `docker-compose.yml` ?
3. Why do we use Docker Compose?

Intermediate

1. Difference between Dockerfile and Docker Compose?
2. How containers communicate in Docker Compose?
3. What is `depends_on` ?
4. How do you persist data in Docker Compose?

Advanced / Scenario

1. Database container restart ho gaya, data kaise safe raha?
2. Multiple environments ke liye Compose kaise use karoge?
3. Compose me app accessible nahi hai, kaise troubleshoot karoge?

One-Line Interview Answer

Docker Compose is a tool used to define and manage multi-container Docker applications using a single YAML configuration file.

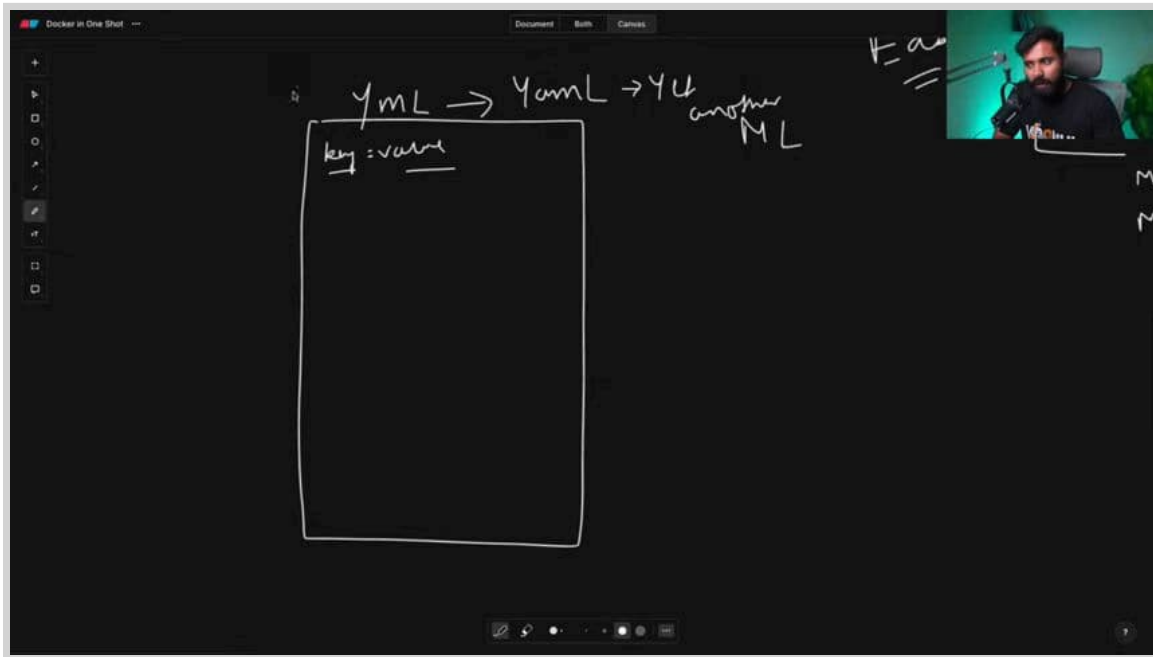
Interviewer Impress Tip

Bolna:

“Docker Compose simplifies running multi-container applications by managing services, networks, and volumes together.”

docker compose ek yaml base file hai jisme aap key or value paas karte ho

02:01:05



02:02:06



docker-compose ka current verison jo currently chal rha hai abhi versoin: "3.8" chal rha hai

====> docker-compose file ko run karne ke liye pahale docker-compose ka latest version install karna padega

02:09:06

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ vim docker-c
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ sudo apt-get
r-compose-v2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  docker-compose-v2
0 upgraded, 1 newly installed, 0 to remove and 30 not upgraded.
Need to get 12.7 MB of archives.
After this operation, 54.2 MB of additional disk space will be used.
Get:1 http://us-west-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 docke
r-compose-v2 amd64 2.24.6+ds1-0ubuntu2 [12.7 MB]
Fetched 12.7 MB in 1s (18.0 MB/s)
```

docker-compose ki file create kake

simply

docker compose up command run kar do sab kuch automate ho
jaayega

02:09:52

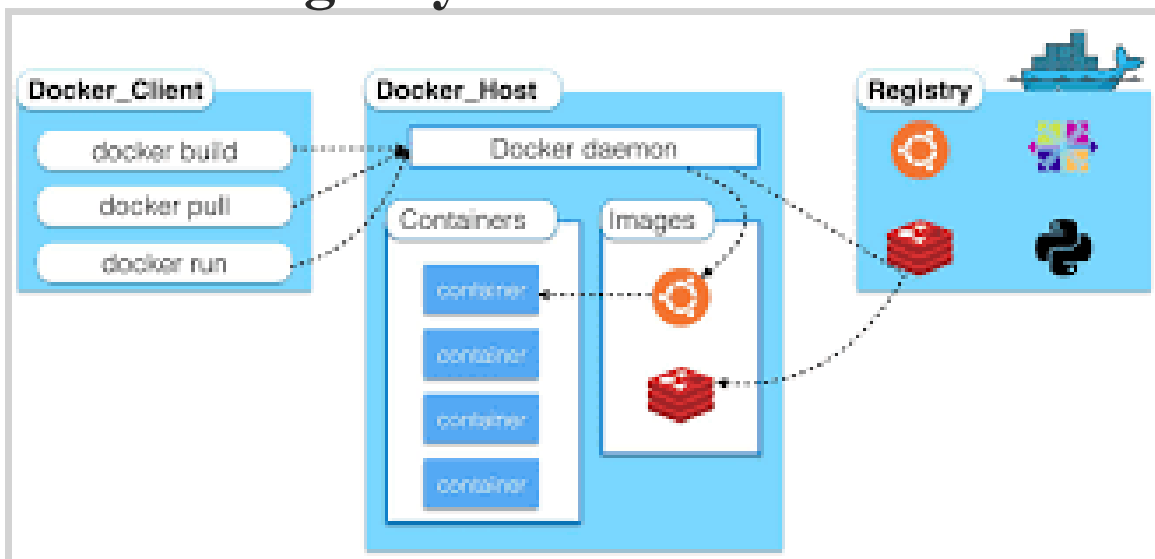
```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ vim docker-c
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker compo
validating /home/ubuntu/projects/two-tier-flask-app/docker-compos
Additional property versoin is not allowed
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ vim docker-compose.yml
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker compose up
```

02:17:32

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker compose up -d
[+] Running 2/2
 ✓ Container mysql          Started
 ✓ Container two-tier-backend Started
```

Docker Registry

Docker Registry



https://www.tutorialspoint.com/docker/images/docker_hub_1.jpg

 https://civo-com-assets.ams3.digitaloceanspaces.com/content_images/2015.blog.png?1671109414=

What is Docker Registry?

A **Docker Registry** is a **central storage system** used to **store, manage, and distribute Docker images**.

It allows us to:

- **Push images** (upload)
- **Pull images** (download)
- Share images across servers and teams

👉 **Docker Registry = Image storage & distribution system**

Types of Docker Registries

1 Public Registry

- Accessible by everyone
- Example: Docker Hub
- Mostly used for open-source images

📌 Example:

```
docker pull nginx
```

2 Private Registry

- Restricted access
- Used by companies for internal images
- More secure

📌 Examples:

- AWS ECR



- GitLab Container Registry
- Harbor
- Self-hosted Registry

Docker Registry Workflow

1. Developer builds Docker image
2. Image is tagged
3. Image is pushed to registry
4. Server pulls image
5. Container is created

Important Docker Registry Commands

```
docker login
docker tag myapp:v1 myrepo/myapp:v1
docker push myrepo/myapp:v1
docker pull myrepo/myapp:v1
```

Docker Registry vs Docker Hub

Docker Registry	Docker Hub	Generic concept	Specific service	Can be private
		Mostly public	Self-hosted possible	Cloud only
		Used in enterprises	Used by individuals	

Self-Hosted Docker Registry (Interview Bonus)

You can run your own registry:

```
docker run -d -p 5000:5000 --name registry
registry:2
```

Access:

```
localhost:5000/myimage:v1
```

Security in Docker Registry

- Authentication & authorization
- TLS encryption
- Private repositories
- Image vulnerability scanning

Real-World Example (Interview GOLD)

In our organization, we used a private Docker registry to store internal application images.

CI/CD pipelines pushed images to the registry, and production servers pulled images securely.

INTERVIEW QUESTIONS (VERY IMPORTANT)

Basic

1. What is Docker Registry?
2. Difference between Docker Registry and Docker Hub?
3. Why do we need private registry?

Intermediate

1. How do you push image to private registry?
2. What is image tagging?
3. How do you secure Docker Registry?

Advanced / Scenario

1. Production server cannot pull image from registry. How will you troubleshoot?
2. Image pulled but container not starting. What next?
3. How do you clean old images from registry?

One-Line Interview Answer

A Docker Registry is a service used to store, manage, and distribute Docker images.

Interviewer Impress Tip

Bolna:

“In production, we prefer private Docker registries for security, access control, and better CI/CD integration.”

==> in single command sabhi container inactive or netowrks

[02:20:50](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker system prune
WARNING! This will remove:
- all stopped containers
- all networks not used by at least one container
- all dangling images
- all dangling build cache

Are you sure you want to continue? [y/N]
```

remove docker images

docker rmi command

or docker rmi -a

[02:22:25](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker image
56ae212eeec7
be960704dfac
d2c94e258dcb
264c9bdce361
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker rmi -f $(docker ima
ges -aq)
```

stops containers ko remove karna hai

to remove karne ke liye docker compose ki command hai

[02:23:14](#)

```
run          Run a one-off command on a service
scale        Scale services
start        Start services
stats        Display a live stream of container(s) resource usage
stop         Stop services
top          Display the running processes
unpause      Unpause services
up           Create and start containers
version      Show the Docker Compose version information
wait         Block until the first service container stops
watch        Watch build context for service and rebuild/refresh containers wh
en files are updated

Run 'docker compose COMMAND --help' for more information on a command.
unknown docker command: "compose remove"
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker compose down
[+] Running 0/1
    2.6s Container two-tier-backend Stopping
```

[02:24:16](#)

```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker images
REPOSITORY      TAG              IMAGE ID         CREATED          SIZE
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker compose up -d --build
ld
[+] Running 0/1
  * mysql 10 layers [          ] 0B/0B      Pulling      1.2s
  * eba3c26198b7 Pulling fs layer
  * 97f7c8c33abe Pulling fs layer
  * aa23d877fa04 Pulling fs layer
  * a143609ddd2d Waiting
  * 78308a3437c4 Waiting
  * c0880e4b3737 Waiting
  * 4bab267f9ce1 Waiting
  * e575f6d9b17a Waiting
  * 607f86c00053 Waiting
  * cd68caa5febe Waiting
  * 0.1s
```

02:25:46

```
ubuntu@ip-172-31-7-73:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
bc228892034a   flask-app     "python run.py"         5 minutes ago Up 16 seconds
->80/tcp, :::80->80/tcp   serene_mestorf
2fcba592fbb2   mysql        "docker-entrypoint.s..." 32 minutes ago Up 32 minutes 3306/tcp,
33060/tcp      funny_turing
ubuntu@ip-172-31-7-73:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
85710ecd93b0   two-tier-flask-app-flask "python app.py"         51 seconds ago Up 46 seconds
ds (health: starting) 0.0.0.0:5000->5000/tcp, :::5000->5000/tcp two-tier-backend
e172f08ccfd8   mysql        "docker-entrypoint.s..." 51 seconds ago Up 50 seconds
ds (healthy) 0.0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp mysql
ubuntu@ip-172-31-7-73:~$ cd projects/two-tier-flask-app/
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ ls
Dockerfile      Makefile  docker-compose.yml  message.sql
Dockerfile-multistage README.md  eks-manifests       requirements.txt
Jenkinsfile     app.py    k8s                 templates
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker compose down
[+] Running 0/1
  * Container two-tier-backend Stopping      8.0s
```

02:27:45


```
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker image
REPOSITORY          TAG          IMAGE ID          CREATED
two-tier-flask-app-flask  latest      19710a048c06     About a minute
mysql                latest      be960704dfac     3 weeks ago
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker login
Authenticating with existing credentials...
WARNING! Your password will be stored unencrypted in /home/ubuntu/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker image tag two-tier-
flask-app-flask:latest trainwithshubham/two-tier-backend:latest
```

02:28:14

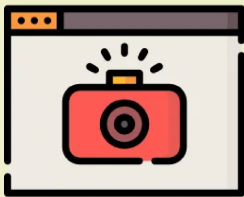
```
1MB
trainwithshubham/two-tier-backend  latest      19710a048c06  3 mi
1MB
mysql                               latest      be960704dfac  3 we
2MB
ubuntu@ip-172-31-7-73:~/projects/two-tier-flask-app$ docker push trainwithshubh
am/two-tier-backend:latest
The push refers to repository [docker.io/trainwithshubham/two-tier-backend]
6820c2e3fc9b: Preparing
bbed6f388614: Preparing
1e1d5ca693c8: Preparing
fe40adaecfe4: Preparing
888d56f24a6c: Preparing
6b3bf59a8be1: Waiting
d86feaf80e98: Waiting
19f5accf4683: Waiting
0300a07ea341: Waiting
98b5f35ea9d3: Waiting
```

multi stage docker build



Remove Ads from pdf and websites

Pricing



Now you can use Askify in any websites

See How